

```
!pip install yfinance
!pip install tensorflow
!pip install scikit-learn
!pip install faiss-cpu
!pip install sentence-transformers
!pip install transformers torch
!pip install groq
!pip install pdfplumber
!pip install requests
!pip install pandas
!pip install numpy
!pip install matplotlib
```

```
!pip install scipy
!pip install xgboost
!pip install lightgbm
!pip install optuna
!pip install statsmodels
!pip install prophet
!pip install pmdarima
!pip install textblob
!pip install beautifulsoup4
!pip install ta
!pip install plotly
!pip install seaborn
```

[Show hidden output](#)

```
# 2. api key
from groq import Groq
from google.colab import userdata

NEWSDATA_API_KEY = userdata.get('NEWSDATA_API_KEY')
GNEWS_API_KEY    = userdata.get('GNEWS_API_KEY')
GROQ_API_KEY     = userdata.get('GROQ_API_KEY')
GROQ_MODEL       = 'llama-3.1-8b-instant'

print("API keys loaded")
```

API keys loaded

```
#3. get data
company_name = input("Enter company name (e.g. Tata Consumer): ")
ticker       = input("Enter NSE ticker (e.g. TATACONSUM.NS): ")

print(f"\nCompany : {company_name}")
print(f"Ticker   : {ticker}")
print("Ready to fetch data ✓")
```

Enter company name (e.g. Tata Consumer): Tata Consumer
Enter NSE ticker (e.g. TATACONSUM.NS): TATACONSUM.NS

Company : Tata Consumer
Ticker : TATACONSUM.NS
Ready to fetch data ✓

```
import requests
headlines = []

# Smart query builder (strips .NS / .BO suffix)
def build_query(company_name, ticker):
    clean_ticker = ticker.replace(".NS", "").replace(".BO", "").strip()
    if company_name and company_name != clean_ticker:
        return f"{company_name} stock"
    return f"{clean_ticker} stock India"

# Helper: clean & duplicate
def clean_headlines(raw_list):
    seen = set()
    out = []
    for h in raw_list:
        h = h.strip()
        if h and h not in seen:
```

```

        seen.add(h)
        out.append(h)
    return out

# newdata.io
def fetch_newdata(query, api_key, max_results=10):
    results = []
    try:
        url = "https://newdata.io/api/1/news"
        params = {
            "apikey": api_key,
            "q": query,
            "language": "en",
            "category": "business",
            "size": max_results
        }
        r = requests.get(url, params=params, timeout=15)
        r.raise_for_status()
        data = r.json()

        print(f" [newdata.io] status : {data.get('status')} | "
              f"total: {data.get('totalResults', 'N/A')}")

        if data.get("status") == "success":
            for article in data.get("results", []):
                title = article.get("title") or article.get("description") or ""
                if title:
                    results.append(title)
            else:
                print(f" [newdata.io] API error: {data}")
    except requests.exceptions.HTTPError as e:
        print(f" [newdata.io] HTTP {e.response.status_code}: {e.response.text[:300]}")
    except Exception as e:
        print(f" [newdata.io] Exception: {e}")
    return results

# GNews
def fetch_gnews(query, api_key, max_results=10):
    results = []
    try:
        url = "https://gnews.io/api/v4/search"
        params = {
            "token": api_key,
            "q": query,
            "lang": "en",
            "country": "in",
            "max": max_results,
            "sortby": "publishedAt"
        }
        r = requests.get(url, params=params, timeout=15)
        r.raise_for_status()
        data = r.json()

        articles = data.get("articles", [])
        print(f" [GNews] articles: {len(articles)}")
        for article in articles:
            title = article.get("title") or article.get("description") or ""
            if title:
                results.append(title)
        if not articles:
            print(f" [GNews] Full response: {data}")
    except requests.exceptions.HTTPError as e:
        print(f" [GNews] HTTP {e.response.status_code}: {e.response.text[:300]}")
    except Exception as e:
        print(f" [GNews] Exception: {e}")
    return results

# Build search query
search_query = build_query(company_name, ticker)
print(f"Query will use: '{search_query}'\n")

Query will use: 'Tata Consumer stock'

```

```

#menu
print("Available news sources:")
print(" 1. newdata.io only")

```

```

print(" 2. GNews only")
print(" 3. Both (recommended)")

choice = input("\nEnter choice (1/2/3): ").strip()
if choice == "1":
    print("\n Fetching from newsdata.io...\n")
    nd_headlines = fetch_newsdata(search_query, NEWSDATA_API_KEY)
    gn_headlines = []
elif choice == "2":
    print("\n Fetching from GNews...\n")
    nd_headlines = []
    gn_headlines = fetch_gnews(search_query, GNEWS_API_KEY)
elif choice == "3":
    print("\n Fetching from both sources...\n")
    nd_headlines = fetch_newsdata(search_query, NEWSDATA_API_KEY)
    gn_headlines = fetch_gnews(search_query, GNEWS_API_KEY)
else:
    print("\n Invalid choice – defaulting to Both...\n")
    nd_headlines = fetch_newsdata(search_query, NEWSDATA_API_KEY)
    gn_headlines = fetch_gnews(search_query, GNEWS_API_KEY)
# merge & deduplicate
headlines = clean_headlines(nd_headlines + gn_headlines) # used by later cells
print(f" newsdata.io : {len(nd_headlines)} headlines")
print(f" GNews : {len(gn_headlines)} headlines")
print(f" Total unique : {len(headlines)} headlines")
print("Headlines:")
for i, h in enumerate(headlines, 1):
    print(f" {i}. {h}")
if not headlines:
    print(" 0 results – paste the debug output above for diagnosis.")

```

Available news sources:

1. newsdata.io only
2. GNews only
3. Both (recommended)

Enter choice (1/2/3): 3

Fetching from both sources...

```

[newsdata.io] status : success | total: 75
[GNews] articles: 2
newsdata.io : 10 headlines
GNews : 2 headlines
Total unique : 12 headlines

```

Headlines:

1. Stock Market Outlook Today: Will Dalal Street Recover After 1,300-Point Crash? Here's What Analysts Predict Ahead Of Openin
2. Tata Consumer Share Price Live Updates: Tata Consumer's beta insights
3. Nifty Weekly Expiry Today: Index Tests Key 23,800 Support
4. Stocks To Watch Today (May 12, 2026): From JSW Energy To Swiggy, Key Earnings, Orders And Deals That Could Move The Market
5. Stocks to watch: Tata Power, Dixon Tech, Indian Hotels among 10 shares in focus today
6. Ruchir Sharma at Express Adda: 'Asia, Europe need coordinated strategy to survive US or will be eaten for lunch'
7. Parag Parikh Flexi Cap reduces cash holding to 15.5%, increases stakes in ITC, TCS and Infosys
8. Stocks to Watch Today: Indian Hotels, Syrma SGS, Groww, Afcons Infra, Bharat Forge, HG Infra, Munjal Auto, HFCL, Windsor Ma
9. Stocks in news: Groww, Dr Reddy's, Waaree Energies, Lenskart, SBI
10. Q4 Results Today Highlights: Canara Bank Q4 profit dips, UPL, PVR Inox, Shyam Metalics profit rise, JSW Energy, IHCL, Abbo
11. Tata Group stock surges the most in six years after strong Q4 results, future outlook
12. Raja Venkatraman, MarketSmith recommend five stocks for 30 April

rag

```

import requests, os, time
from bs4 import BeautifulSoup

# Use the TICKER, clean the .NS or .BO
TICKER = ticker.replace(".NS", "").replace(".BO", "").strip()
PDF_PATH = f"/content/{TICKER}_annual_report.pdf"

HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36"
}

def get_screener_pdf_url(ticker_symbol):
    url = f"https://www.screener.in/company/{ticker_symbol}/"
    try:
        r = requests.get(url, headers=HEADERS, timeout=15)
        r.raise_for_status()

```

```

    soup = BeautifulSoup(r.text, "html.parser")

    # Look for the first link under "Annual reports" pointing to a PDF
    for a in soup.find_all("a", href=True):
        href = a["href"]
        text = a.text.strip().lower()
        if "financial year" in text and href.lower().endswith(".pdf"):
            return href
    except Exception as e:
        print(f"Failed to scrape Screener: {e}")
    return None

def download_pdf(pdf_url, save_path):
    r = requests.get(pdf_url, headers=HEADERS, timeout=60, stream=True)
    r.raise_for_status()
    with open(save_path, "wb") as f:
        for chunk in r.iter_content(chunk_size=8192):
            f.write(chunk)
    return os.path.getsize(save_path) / 1024

# MAIN
print(f" Fetching latest annual report for: {TICKER}\n")

pdf_url = get_screener_pdf_url(TICKER)

if pdf_url:
    print(f" Found Report URL! Downloading: {pdf_url}")
    size_kb = download_pdf(pdf_url, PDF_PATH)
    print(f" PDF saved → {PDF_PATH} ({size_kb:.0f} KB)")
else:
    print(f"\n Automated scrape failed for '{TICKER}'.")
    print(f" Please go to https://www.screener.in/company/{TICKER}/")
    print(f" Scroll down to 'Documents' , 'Annual reports' and copy the latest PDF link.")

# Optional Manual override
manual_url = input("Paste the PDF link here (or hit enter to skip): ").strip()

if manual_url:
    print(f" Downloading manual URL: {manual_url}")
    size_kb = download_pdf(manual_url, PDF_PATH)
    print(f" PDF saved → {PDF_PATH} ({size_kb:.0f} KB)")
else:
    print("\n No URL provided. Skipping PDF download.")

```

Fetching latest annual report for: TATACONSUM

Found Report URL! Downloading: <https://www.bseindia.com/xml-data/corpfilings/AttachHis/fcd18b51-5f3f-4fc8-950a-b1ca424b85b2.pdf>
 PDF saved → /content/TATACONSUM_annual_report.pdf (20775 KB)

```

import os
if os.path.exists(f"/content/{TICKER}_pdf_summary.txt"):
    os.remove(f"/content/{TICKER}_pdf_summary.txt")
    print("Cleared old summary – ready to restart")

```

```

!pip install pdfplumber groq --quiet

import os, pdfplumber
from groq import Groq

# CONFIG
MAX_PAGES = 20 # only first 50 pages
MAX_CHUNK_CHARS = 20000 # bigger chunks = fewer API calls
GROQ_API_KEY = userdata.get('GROQ_API_KEY') # ADD THIS
client = Groq(api_key=GROQ_API_KEY)
print("Groq client initialized")

# EXTRACT TEXT
def extract_pdf_text(pdf_path: str) -> str:
    print(f"Extracting text from: {pdf_path}")
    pages = []
    with pdfplumber.open(pdf_path) as pdf:
        total = min(MAX_PAGES, len(pdf.pages))
        for i, page in enumerate(pdf.pages):
            if i >= MAX_PAGES:
                break

```

```

        text = page.extract_text()
        if text:
            pages.append(text)
        if (i + 1) % 10 == 0:
            print(f"    ... read {i+1}/{total} pages")
    full_text = "\n\n".join(pages)
    print(f" Extracted {len(full_text):,} characters from {total} pages\n")
    return full_text

# CHUNK TEXT
def chunk_text(text: str) -> list:
    chunks, current = [], ""
    for paragraph in text.split("\n\n"):
        if len(current) + len(paragraph) > MAX_CHUNK_CHARS:
            if current:
                chunks.append(current.strip())
                current = paragraph
            else:
                current += "\n\n" + paragraph
        if current.strip():
            chunks.append(current.strip())
    print(f" Split into {len(chunks)} chunks\n")
    return chunks

# SUMMARISE
CHUNK_PROMPT = """You are a financial analyst. Summarize this annual report section.
Focus on: revenue, profit, risks, business highlights, management outlook.
Be concise. Use bullet points.

SECTION:
{chunk}
"""

FINAL_PROMPT = """You are a financial analyst. Combine these section summaries of
{company}'s annual report into ONE executive summary covering:
1. Financial Performance
2. Business Highlights
3. Risks
4. Management Outlook

SUMMARIES:
{summaries}
"""

def summarize_chunk(chunk: str, index: int, total: int) -> str:
    print(f" Summarizing chunk {index+1}/{total} ({len(chunk):,} chars)...")
    response = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[{"role": "user", "content": CHUNK_PROMPT.format(chunk=chunk)}],
        temperature=0.2,
        max_tokens=512,
    )
    return response.choices[0].message.content.strip()

def final_summary(chunk_summaries: list, company: str) -> str:
    print(f"\n Generating final summary for {company}...")
    combined = "\n\n---\n\n".join(
        f"[Section {i+1}]\n{s}" for i, s in enumerate(chunk_summaries)
    )
    response = client.chat.completions.create(
        model=GROQ_MODEL,
        messages=[{"role": "user", "content": FINAL_PROMPT.format(
            company=company, summaries=combined
        )}],
        temperature=0.2,
        max_tokens=1024,
    )
    return response.choices[0].message.content.strip()

SUMMARY_PATH = f"/content/{TICKER}_pdf_summary.txt"

# RELOAD
if os.path.exists(SUMMARY_PATH):
    print(f" Summary already exists: loading from file (skipping Groq API calls)\n")
    with open(SUMMARY_PATH, "r") as f:
        PDF_SUMMARY = f.read()
    print(f" Loaded summary ({len(PDF_SUMMARY):,} characters)")

```

```

else:

    print(f" Summarizing annual report for: {TICKER}")

    if not os.path.exists(PDF_PATH):
        print(f" PDF not found at: {PDF_PATH}")
        print(" Please re-run first.")
    else:
        raw_text      = extract_pdf_text(PDF_PATH)
        chunks        = chunk_text(raw_text)

        print(" Sending chunks to Groq...\n")
        chunk_summaries = [
            summarize_chunk(chunk, i, len(chunks))
            for i, chunk in enumerate(chunks)
        ]

        PDF_SUMMARY = final_summary(chunk_summaries, TICKER)
        with open(SUMMARY_PATH, "w") as f:
            f.write(PDF_SUMMARY)

        print(f"\n Summary saved → {SUMMARY_PATH}")

print(PDF_SUMMARY)

```

```

Groq client initialized
Summary already exists: loading from file (skipping Groq API calls)

Loaded summary (2,221 characters)
**Executive Summary: Tata Consumer Products Annual Report**

**Financial Performance:**

Tata Consumer Products has delivered a strong financial performance, with consolidated revenue of INR 99,000 Crores for FY 2024-

**Business Highlights:**

Tata Consumer Products has grown into a multi-category premier FMCG Company, with a strong foundation and capabilities to power

**Risks:**

The company faces risks related to global economic uncertainty, changes in consumer preferences, and supply chain disruptions. H

**Management Outlook:**

The company's vision is to build better lives and thriving communities, and its mission is to support its FMCG ambitions and eve

```

```

CHUNK_SIZE      = 500
CHUNK_OVERLAP   = 100
SUMMARY_PATH    = f"/content/{TICKER}_pdf_summary.txt"

if not os.path.exists(SUMMARY_PATH):
    print("Summary file not found. Please run the pdf summary cell first.")
else:
    with open(SUMMARY_PATH, "r") as f:
        PDF_SUMMARY = f.read()
    print(f"Loaded summary: {len(PDF_SUMMARY):,} characters")

def chunk_summary(text: str, chunk_size: int, overlap: int) -> list:
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end].strip())
        start += chunk_size - overlap # move forward but keep overlap
    chunks = [c for c in chunks if len(c) > 50]
    return chunks

```

```

Loaded summary: 2,221 characters

```

```

PDF_CHUNKS = chunk_summary(PDF_SUMMARY, CHUNK_SIZE, CHUNK_OVERLAP)
print(f"Total chunks created : {len(PDF_CHUNKS)}")
print(f"Chunk size           : {CHUNK_SIZE} characters")
print(f"Overlap                 : {CHUNK_OVERLAP} characters")
print()

```

```
for i, chunk in enumerate(PDF_CHUNKS[:3]):
    print(f" Chunk {i+1} ")
    print(chunk)
    print()
```

```
Total chunks created : 6
Chunk size           : 500 characters
Overlap              : 100 characters
```

```
Chunk 1
**Executive Summary: Tata Consumer Products Annual Report**
```

```
**Financial Performance:**
```

Tata Consumer Products has delivered a strong financial performance, with consolidated revenue of INR 99,000 Crores for FY 2024-

```
Chunk 2
t of INR 1,416 Crores, with an EBITDA margin of 10.8% and a ROCE of 37.5%. The company's revenue growth has been consistent, wit
```

```
**Business Highlights:**
```

Tata Consumer Products has grown into a multi-category premier FMCG Company, with a strong foundation and capabilities to power

```
Chunk 3
ny has expanded its reach, built scalable platforms, and developed future-ready capabilities. Its brand portfolio now spans a ga
```

```
**Risks:**
```

The company faces risks related to global economic uncertainty, changes in consumer preferences, a

```
# Build Knowledge Base
documents = []

# Add PDF chunks
if PDF_CHUNKS:
    documents.extend(PDF_CHUNKS)
    print(f'PDF chunks added      : {len(PDF_CHUNKS)}')
else:
    print('No PDF chunks found')

# Add news headlines
if headlines:
    documents.extend(headlines)
    print(f'Headlines added          : {len(headlines)}')
else:
    print('No headlines found')

print(f'\nTotal documents in knowledge base : {len(documents)}')
print(f'Preview of first 2:')
for i, d in enumerate(documents[:2]):
    print(f' [{i+1}] {d[:100]}')
```

```
PDF chunks added      : 6
Headlines added        : 12
```

```
Total documents in knowledge base : 18
```

```
Preview of first 2:
```

```
[1] **Executive Summary: Tata Consumer Products Annual Report**
```

```
**Financial Performance:**
```

```
Tata Consum
```

```
[2] t of INR 1,416 Crores, with an EBITDA margin of 10.8% and a ROCE of 37.5%. The company's revenue gro
```

```
# FAISS Embeddings
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('all-MiniLM-L6-v2')

# Embed all documents
vecs = embedder.encode(documents, normalize_embeddings=True).astype(np.float32)

# Build FAISS index
```

```
index = faiss.IndexFlatIP(vecs.shape[1])
index.add(vecs)
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 22.7kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.83kB/s]

README.md: 10.5k/? [00:00<00:00, 665kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.25kB/s]

config.json: 100% 612/612 [00:00<00:00, 51.6kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 457MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 489.64it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 14.2kB/s]

vocab.txt: 232k/? [00:00<00:00, 6.08MB/s]

tokenizer.json: 466k/? [00:00<00:00, 13.0MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 6.89kB/s]

config.json: 100% 190/190 [00:00<00:00, 8.67kB/s]
```

```
print(f'Embedding model : all-MiniLM-L6-v2')
print(f'Documents embedded : {len(documents)}')
print(f'Vector dimension : {vecs.shape[1]}')
print(f'FAISS index size : {index.ntotal}')
```

```
Embedding model : all-MiniLM-L6-v2
Documents embedded : 18
Vector dimension : 384
FAISS index size : 18
```

```
# Groq RAG
import re, json
from groq import Groq
groq_client = Groq(api_key=GROQ_API_KEY)
def ask_rag(question):

    # find most relevant documents
    q_vec = embedder.encode([question], normalize_embeddings=True).astype(np.float32)
    _, ids = index.search(q_vec, k=3)
    context = '\n\n'.join([documents[i] for i in ids[0] if i < len(documents)])

    # Groq answers using that context
    resp = groq_client.chat.completions.create(
        model='llama-3.1-8b-instant',
        messages=[
            {'role': 'system', 'content':
                f'You are a stock analyst for {company_name}. '
                f'Answer only using the context provided. Be concise.'},
            {'role': 'user', 'content':
                f'Context:\n{context}\n\nQuestion: {question}'}
        ],
        temperature=0.2,
        max_tokens=300
    )
    return resp.choices[0].message.content.strip()
```

```
# Get RAG score from PDF/Screeener data
try:
```

```

rag_question = "Is the company outlook bullish or bearish? What are the key risks and growth signals?"
rag_answer = ask_rag(rag_question)
print(f' RAG Answer: {rag_answer}')

# Score based on keywords in answer
answer_lower = rag_answer.lower()
bullish_words = ['growth', 'positive', 'strong', 'bullish', 'increase', 'profit', 'revenue']
bearish_words = ['risk', 'decline', 'bearish', 'negative', 'loss', 'weak', 'concern']

bull_count = sum(1 for w in bullish_words if w in answer_lower)
bear_count = sum(1 for w in bearish_words if w in answer_lower)

rag_score = float(np.clip((bull_count - bear_count) / (bull_count + bear_count + 1e-9), -1, 1))
print(f' RAG score : {rag_score:+.3f}')
except Exception as e:
    rag_score = 0.0
    print(f' RAG error: {e}')

```

RAG Answer: The company outlook is bullish. Key growth signals include:

1. Diversification of product portfolio and expansion of international presence.
2. Significant investments in digital platforms and e-commerce, driving growth and improving customer engagement.
3. Focus on sustainability and responsible business practices.

Key risks include:

1. Global economic uncertainty.
2. Changes in consumer preferences.
3. Supply chain disruptions.
4. Ability to continue delivering strong revenue growth and improving profitability.

RAG score : +0.667

```

print('\n RAG Chatbot ready! Type "exit" to quit.\n')
while True:
    question = input('user: ').strip()
    if question.lower() == 'exit':
        print('chatbot closed.')
        break
    if not question:
        continue
    answer = ask_rag(question)
    print(f'ans: {answer}')
    print()

```

RAG Chatbot ready! Type "exit" to quit.

user: what is the current price

ans: Unfortunately, the current price of Tata Consumer is not mentioned in the provided context. However, I can suggest checking

user: give recent news

ans: Based on the context, here are some recent news updates:

1. ****Canara Bank Q4 profit dips****: The bank's Q4 profit has seen a decline, which may impact investor sentiment.
2. ****UPL, PVR Inox, Shyam Metalics profit rise****: These companies have reported a rise in profits, which could be a positive sig
3. ****JSW Energy, IHCL, Abbott, JB Chemicals, Nuvama, Syrma SGS Tech, Anant Raj, JBM Auto to announce Q4 results****: Several compa
4. ****Indian Hotels, Syrma SGS, Groww, Afcons Infra, Bharat Forge, HG Infra, Munjal Auto, HFCL, Windsor Machines in focus****: Thes

user: what is the stock price

ans: As a stock analyst for Tata Consumer, I can provide you with the current stock price of Tata Consumer. However, it's not di

To provide an accurate answer, I would need access to real-time market data. Nevertheless, I can suggest checking the current st

user: exit

chatbot closed.

```

# News Sentiment (TextBlob + FinBERT)
from textblob import TextBlob

# TextBlob sentiment on headlines
tb_scores = [TextBlob(h).sentiment.polarity for h in headlines]
textblob_score = float(np.mean(tb_scores)) if headlines else 0.0
print(f' TextBlob avg senti9ment: {textblob_score:+.4f}')

# FinBERT
finbert_score = 0.0
print('\nLoading FinBERT...')
try:

```

```

from transformers import pipeline
finbert = pipeline('text-classification', model='ProsusAI/finbert',
                   top_k=None, truncation=True, max_length=512)
LABEL_MAP = {'positive': 1, 'neutral': 0, 'negative': -1}
scores = []
for h in headlines[:10]:
    res = finbert(h)[0]
    w = sum(LABEL_MAP.get(r['label'].lower(), 0) * r['score'] for r in res)
    scores.append(w)
finbert_score = float(np.mean(scores))
label = 'POSITIVE' if finbert_score > 0.1 else 'NEGATIVE' if finbert_score < -0.1 else 'NEUTRAL'
print(f' FinBERT score: {finbert_score:+.4f} -> {label}')
except Exception as e:
    finbert_score = textblob_score
    print(f' FinBERT unavailable ({e}), using TextBlob fallback')

# RAG score from PDF (already computed in Cell 13)
# If PDF RAG was skipped, default to 0
if 'rag_score' not in dir():
    rag_score = 0.0

print(f'\n textblob_score : {textblob_score:+.4f}')
print(f' finbert_score   : {finbert_score:+.4f}')
print(f' rag_score        : {rag_score:+.4f}')

```

TextBlob avg senti9ment: +0.0373

Loading FinBERT...

```

config.json: 100% 758/758 [00:00<00:00, 12.2kB/s]

pytorch_model.bin: 100% 438M/438M [00:06<00:00, 104MB/s]

model.safetensors: 100% 438M/438M [00:09<00:00, 80.9MB/s]

Loading weights: 100% 201/201 [00:02<00:00, 82.53it/s, Materializing param=classifier.weight]
BertForSequenceClassification LOAD REPORT from: ProsusAI/finbert
Key | Status | |
-----+-----+--
bert.embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 252/252 [00:00<00:00, 1.83kB/s]

vocab.txt: 232k/? [00:00<00:00, 874kB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 1.58kB/s]

FinBERT score: +0.0107 -> NEUTRAL

textblob_score : +0.0373
finbert_score   : +0.0107
rag_score        : +0.6667

```

```

from datetime import datetime
import yfinance as yf
import matplotlib.pyplot as plt

# Take input from user
start_date = input("Enter start date (YYYY-MM-DD): ")
end_date = input("Enter end date (YYYY-MM-DD): ")

# Validate format (optional but good practice)
try:
    datetime.strptime(start_date, "%Y-%m-%d")
    datetime.strptime(end_date, "%Y-%m-%d")
except ValueError:
    print("Invalid date format! Use YYYY-MM-DD")
    exit()

print(f"Fetching data from {start_date} to {end_date}...")

# Fetch data
data = yf.download(ticker, start=start_date, end=end_date)

# Keep only Close price
data = data[['Close']].dropna()

# Plot

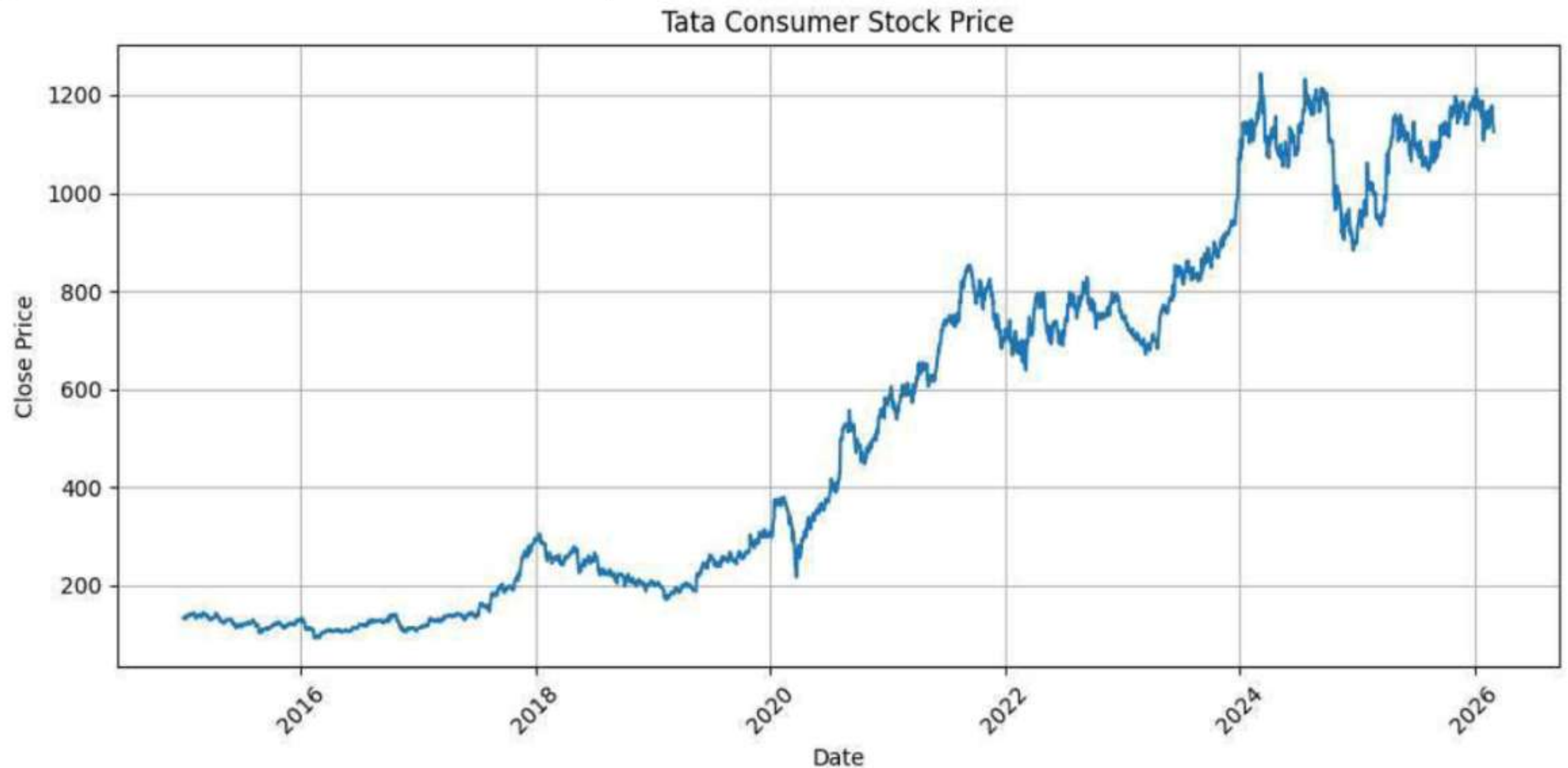
```

```
plt.figure(figsize=(10, 5))
plt.plot(data.index, data['Close'])

plt.title(f"{company_name} Stock Price")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.xticks(rotation=45)
plt.grid()

plt.tight_layout()
plt.show()
```

```
Enter start date (YYYY-MM-DD): 2015-01-01
Enter end date (YYYY-MM-DD): 2026-03-03
Fetching data from 2015-01-01 to 2026-03-03...
/tmp/ipykernel_7774/1232137925.py:20: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download(ticker, start=start_date, end=end_date)
[*****100%*****] 1 of 1 completed
```



```
# 4. Fetch Stock Data ; Fetch last 5 years of data dynamically
end_date = datetime.today().strftime('%Y-%m-%d')
print(f"Fetching data for {company_name} ({ticker})...")
data = yf.download(ticker, start=start_date, end=end_date)

# Keep only Close price
data = data[['Close']].dropna()
print(f"Data fetched from {start_date} to {end_date}")
print(f"Total trading days : {len(data)}")
print(data.tail())
```

```
Fetching data for Tata Consumer (TATACONSUM.NS)...
/tmp/ipykernel_7774/1339614841.py:4: FutureWarning: YF.download() has changed argument auto_adjust default to True
  data = yf.download(ticker, start=start_date, end=end_date)
[*****100%*****] 1 of 1 completedData fetched from 2015-01-01 to 2026-05-12
Total trading days : 2803
Price          Close
Ticker         TATACONSUM.NS
Date
2026-05-05    1153.500000
2026-05-06    1152.199951
2026-05-07    1151.699951
2026-05-08    1176.199951
2026-05-11    1271.000000
```

```
import pandas as pd
```

```
from prophet import Prophet
from sklearn.preprocessing import RobustScaler
```

```

print(f'Fetching OHLCV for {ticker}...')
raw = yf.download(ticker, start=start_date, end=end_date,
                  auto_adjust=True, progress=False)

if isinstance(raw.columns, pd.MultiIndex):
    raw.columns = raw.columns.get_level_values(0)

ohlc = raw[['Open', 'High', 'Low', 'Close', 'Volume']].copy()
ohlc.index = pd.to_datetime(ohlc.index).tz_localize(None)
ohlc.dropna(inplace=True)

if ohlc.empty:
    raise SystemExit(f'ERROR: No data returned for {ticker}. Check the ticker symbol and date format (YYYY-MM-DD).')

print(f' {len(ohlc)} trading days '
      f'({ohlc.index[0].date()} -> {ohlc.index[-1].date()})')
print(f' Latest Close: Rs.{ohlc["Close"].iloc[-1]:.2f}')

# Macro proxies
print('\nFetching macro data...')
macro_syms = {'^NSEI': 'Nifty50', 'USDINR=X': 'USDINR', 'GC=F': 'Gold'}
macro_df = pd.DataFrame()
for sym, name in macro_syms.items():
    try:
        m = yf.download(sym, start=start_date, end=end_date,
                        auto_adjust=True, progress=False)
        if isinstance(m.columns, pd.MultiIndex):
            m.columns = m.columns.get_level_values(0)
        s = m['Close'].rename(name)
        s.index = pd.to_datetime(s.index).tz_localize(None)
        macro_df[name] = s
        print(f' {name}: {len(s)} rows')
    except Exception as e:
        print(f' {name}: {e}')

macro_df = macro_df.reindex(ohlc.index, method='ffill').fillna(method='bfill')
print(f' Macro shape: {macro_df.shape}')

```

```

Fetching OHLCV for TATACONSUM.NS...
2803 trading days (2015-01-01 -> 2026-05-11)
Latest Close: Rs.1271.00

Fetching macro data...
Nifty50: 2793 rows
USDINR: 2955 rows
Gold: 2854 rows
Macro shape: (2803, 3)
/tmp/ipykernel_7774/3244112458.py:36: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version
macro_df = macro_df.reindex(ohlc.index, method='ffill').fillna(method='bfill')

```

```

# FEATURE ENGINEERING

df = ohlc.copy()

# Returns
df['Returns'] = df['Close'].pct_change()
df['Log_Returns'] = np.log(df['Close'] / df['Close'].shift(1))

# RSI
def compute_rsi(close, period=14):
    delta = close.diff()
    gain = delta.clip(lower=0)
    loss = (-delta).clip(lower=0)
    avg_g = gain.ewm(alpha=1/period, min_periods=period).mean()
    avg_l = loss.ewm(alpha=1/period, min_periods=period).mean()
    rs = avg_g / avg_l.replace(0, np.nan)
    return 100 - 100/(1+rs)

df['RSI'] = compute_rsi(df['Close'])

# MACD
ema12 = df['Close'].ewm(span=12, adjust=False).mean()
ema26 = df['Close'].ewm(span=26, adjust=False).mean()
df['MACD'] = ema12 - ema26
df['MACD_Signal'] = df['MACD'].ewm(span=9, adjust=False).mean()
df['MACD_Hist'] = df['MACD'] - df['MACD_Signal']

```

```

# Bollinger Bands
bb_mid = df['Close'].rolling(20).mean()
bb_std = df['Close'].rolling(20).std()
df['BB_Upper'] = bb_mid + 2*bb_std
df['BB_Lower'] = bb_mid - 2*bb_std
df['BB_Width'] = (df['BB_Upper'] - df['BB_Lower']) / bb_mid
df['BB_PctB'] = (df['Close'] - df['BB_Lower']) / (df['BB_Upper'] - df['BB_Lower'] + 1e-9)

# ATR
hl = df['High'] - df['Low']
hc = (df['High'] - df['Close'].shift()).abs()
lc = (df['Low'] - df['Close'].shift()).abs()
df['ATR'] = pd.concat([hl, hc, lc], axis=1).max(axis=1).ewm(span=14, adjust=False).mean()
df['ATR_Pct'] = df['ATR'] / df['Close']

# OBV
df['OBV'] = (np.sign(df['Close'].diff().fillna(0)) * df['Volume']).cumsum()

# Rolling stats
for w in [7, 14, 21, 50]:
    df[f'Roll_Mean_{w}'] = df['Close'].rolling(w).mean()
    df[f'Roll_Std_{w}'] = df['Close'].rolling(w).std()
    df[f'RVol_{w}'] = df['Returns'].rolling(w).std() * np.sqrt(252)

# Momentum
df['Momentum_5'] = df['Close'] / df['Close'].shift(5) - 1
df['Momentum_20'] = df['Close'] / df['Close'].shift(20) - 1

# Spreads
df['HL_Spread'] = (df['High'] - df['Low']) / df['Close']
df['CO_Spread'] = (df['Close'] - df['Open']) / df['Open']

# Stochastic
low14 = df['Low'].rolling(14).min()
high14 = df['High'].rolling(14).max()
df['Stoch_K'] = 100*(df['Close'] - low14) / (high14 - low14 + 1e-9)
df['Stoch_D'] = df['Stoch_K'].rolling(3).mean()

# Williams %R
df['Williams_R'] = -100*(high14 - df['Close']) / (high14 - low14 + 1e-9)

# Volume MA
df['Vol_MA_20'] = df['Volume'].rolling(20).mean()
df['Vol_Ratio'] = df['Volume'] / (df['Vol_MA_20'] + 1)

# Lag features
for lag in [1, 2, 3, 5, 10]:
    df[f'Ret_lag_{lag}'] = df['Returns'].shift(lag)
    df[f'Close_lag_{lag}'] = df['Close'].shift(lag)

# Temporal features
df['DayOfWeek'] = df.index.dayofweek
df['Month'] = df.index.month
df['Quarter'] = df.index.quarter

# Regime detection
ema20 = df['Close'].ewm(span=20, adjust=False).mean()
ema50 = df['Close'].ewm(span=50, adjust=False).mean()
df['Regime'] = np.where(ema20 > ema50, 1, -1)

# Macro returns
for col in macro_df.columns:
    df[f'{col}_Ret'] = macro_df[col].pct_change()

# Outlier treatment
for col in ['Returns', 'Log_Returns']:
    q1, q3 = df[col].quantile(0.01), df[col].quantile(0.99)
    df[col] = df[col].clip(q1, q3)

# Target
df['Target'] = (df['Close'].shift(-1) > df['Close']).astype(int)
df['Next_Close'] = df['Close'].shift(-1)

# Fill and clean
df.ffill(inplace=True)
df.bfill(inplace=True)
df.dropna(inplace=True)

```

```

current_price = df['Close'].iloc[-1]
print(f'Feature matrix: {df.shape}')
print(f'Latest regime: {"BULL" if df["Regime"].iloc[-1]==1 else "BEAR"}')

```

```

Feature matrix: (2803, 58)
Latest regime: BULL

```

```

# Scaling
SEQ_LEN      = 60
TRAIN_RATIO  = 0.80
VAL_RATIO    = 0.10

seq_features = ['Close', 'Volume', 'RSI', 'MACD', 'BB_PctB', 'ATR_Pct']
seq_data     = df[seq_features].values
seq_scaler   = RobustScaler()
scaled_seq   = seq_scaler.fit_transform(seq_data)

price_scaler = RobustScaler()
scaled_close = price_scaler.fit_transform(df['Close'].values.reshape(-1, 1))

def build_sequences(data, close_data, seq_len):
    X, y = [], []
    for i in range(seq_len, len(data)):
        X.append(data[i-seq_len:i])
        y.append(close_data[i, 0])
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)

X_seq, y_seq = build_sequences(scaled_seq, scaled_close, SEQ_LEN)
n            = len(X_seq)
n_tr         = int(n * TRAIN_RATIO)
n_val        = int(n * VAL_RATIO)

X_train = X_seq[:n_tr];          y_train = y_seq[:n_tr]
X_val   = X_seq[n_tr:n_tr+n_val]; y_val   = y_seq[n_tr:n_tr+n_val]
X_test  = X_seq[n_tr+n_val:];     y_test  = y_seq[n_tr+n_val:]

n_features = len(seq_features)
print(f'Sequences: Train={len(X_train)} | Val={len(X_val)} | Test={len(X_test)}')
print(f'Shape: {X_train.shape} (samples, seq_len, features)')

```

```

Sequences: Train=2194 | Val=274 | Test=275
Shape: (2194, 60, 6) (samples, seq_len, features)

```

```

print(df['Target'].value_counts())
print(df['Target'].mean())
print(df[['Close', 'Next_Close', 'Target']].tail(10))

```

```

Target
1    1441
0    1362
Name: count, dtype: int64
0.5140920442383161
Price                Close  Next_Close  Target
Date
2026-04-28  1147.699951  1168.000000         1
2026-04-29  1168.000000  1144.599976         0
2026-04-30  1144.599976  1144.599976         0
2026-05-01  1144.599976  1160.400024         1
2026-05-04  1160.400024  1153.500000         0
2026-05-05  1153.500000  1152.199951         0
2026-05-06  1152.199951  1151.699951         0
2026-05-07  1151.699951  1176.199951         1
2026-05-08  1176.199951  1271.000000         1
2026-05-11  1271.000000  1271.000000         0

```

```

# Market Condition Analysis
recent_regime = df['Regime'].tail(60).mean() # last 60 days
recent_vol    = df['RVol_21'].iloc[-1]
recent_ret    = df['Returns'].tail(60).mean()

if recent_regime < 0 and recent_vol > df['RVol_21'].mean():
    market_condition = 'CHOPPY BEAR'
    condition_note    = 'High volatility + Bearish regime - model accuracy naturally lower'
elif recent_regime < 0:
    market_condition = 'BEAR'
    condition_note    = 'Bearish regime - directional accuracy may be suppressed'

```

```

elif recent_regime > 0 and recent_vol < df['RVol_21'].mean():
    market_condition = 'TRENDING BULL'
    condition_note = 'Low volatility bull trend – models perform best here'
else:
    market_condition = 'BULL'
    condition_note = 'Bullish regime'

print(f'Market Condition : {market_condition}')
print(f'Note : {condition_note}')
print(f'Recent Regime : {recent_regime:.2f} (-1=Bear, +1=Bull)')
print(f'Recent Vol : {recent_vol:.4f} (avg: {df["RVol_21"].mean():.4f})')
print(f'Recent Return : {recent_ret*100:+.3f}% avg daily')

```

```

Market Condition : CHOPPY BEAR
Note : High volatility + Bearish regime – model accuracy naturally lower
Recent Regime : -0.60 (-1=Bear, +1=Bull)
Recent Vol : 0.3311 (avg: 0.2762)
Recent Return : +0.110% avg daily

```

```

# Prophet
print('Running Prophet...')

df_prophet = pd.DataFrame({
    'ds': pd.to_datetime(ohlc.index[:len(df)]),
    'y': (df['Log_Returns'] - df['Log_Returns'].rolling(5).mean()).values
}).reset_index(drop=True)

regressors = ['RSI', 'MACD', 'MACD_Hist', 'BB_Width',
              'ATR_Pct', 'Momentum_5', 'Momentum_20',
              'Vol_Ratio', 'HL_Spread', 'CO_Spread']

for reg in regressors:
    if reg in df.columns:
        df_prophet[reg] = df[reg].values

split_idx = int(len(df_prophet) * 0.60)
train_prophet = df_prophet.iloc[:split_idx]

m_prophet = Prophet(
    changepoint_prior_scale=0.1,
    seasonality_mode='multiplicative',
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False
)
for reg in regressors:
    if reg in train_prophet.columns:
        m_prophet.add_regressor(reg)

m_prophet.fit(train_prophet)
forecast = m_prophet.predict(df_prophet)

test_forecast = forecast.iloc[split_idx:]
test_actual = df_prophet.iloc[split_idx:]
prophet_direction_test = (test_forecast['yhat'].values > 0).astype(int)
actual_direction_test = (test_actual['y'].values > 0).astype(int)
prophet_acc = np.mean(prophet_direction_test == actual_direction_test)

df['Prophet_Return'] = forecast['yhat'].values[:len(df)]
df['Prophet_Trend'] = forecast['trend'].values[:len(df)]
df['Prophet_Upper'] = forecast['yhat_upper'].values[:len(df)]
df['Prophet_Lower'] = forecast['yhat_lower'].values[:len(df)]
df['Prophet_Confidence'] = (1 / (forecast['yhat_upper'].values[:len(df)] - forecast['yhat_lower'].values[:len(df)] + 1e-6))
df['Prophet_Direction'] = (df['Prophet_Return'].shift(-1) > 0).astype(int)

prophet_next_ret = float(df['Prophet_Return'].iloc[-1])
prophet_price = float(current_price * np.exp(prophet_next_ret))

print(f'Prophet Accuracy (test set) : {prophet_acc:.2%}')
print(f'Prophet Next-Day : Rs.{prophet_price:.2f}')

```

```

Running Prophet...
Prophet Accuracy (test set) : 86.63%
Prophet Next-Day : Rs.1365.27

```

```
from pmdarima import auto_arima
```

```
print('Fitting ARIMA / SARIMA')

close_series = df['Close'].copy()
log_close    = np.log(close_series)

TRAIN_END = int(len(log_close) * 0.80)
train_log = log_close.iloc[:TRAIN_END]
test_log  = log_close.iloc[TRAIN_END:]
```

Fitting ARIMA / SARIMA

```
# ARIMA
print(' Auto-ARIMA search')
arima_auto = auto_arima(
    train_log,
    seasonal=False,
    stepwise=True,
    suppress_warnings=True,
    error_action='ignore',
    max_p=5, max_q=5, max_d=2,
    information_criterion='aic'
)
print(f' Best ARIMA order: {arima_auto.order}')

arima_preds_log = arima_auto.predict(n_periods=len(test_log))
arima_preds     = np.exp(arima_preds_log)
arima_actual    = np.exp(test_log.values)

arima_rmse      = np.sqrt(np.mean((arima_preds - arima_actual)**2))
arima_mape      = np.mean(np.abs((arima_preds - arima_actual) / arima_actual)) * 100
arima_dir_acc   = np.mean(np.sign(np.diff(arima_preds)) == np.sign(np.diff(arima_actual)))
arima_acc       = arima_dir_acc

print(f' ARIMA RMSE      : {arima_rmse:.2f}')
print(f' ARIMA MAPE      : {arima_mape:.2f}%')
print(f' ARIMA Dir Acc   : {arima_dir_acc:.2%}')

arima_next_log = float(arima_auto.predict(n_periods=1).iloc[0])
arima_price    = float(np.exp(arima_next_log))
print(f' ARIMA Next-Day : Rs.{arima_price:.2f}')
arima_acc = arima_dir_acc
print(f' ARIMA Accuracy : {arima_acc:.2%}')
```

```
Auto-ARIMA search
Best ARIMA order: (2, 1, 2)
ARIMA RMSE      : 419.34
ARIMA MAPE      : 33.28%
ARIMA Dir Acc   : 49.11%
ARIMA Next-Day : Rs.1103.42
ARIMA Accuracy : 49.11%
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Pr
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. I
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Pr
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. I
return get_prediction_index(
```

```
# SARIMA
print('\n Auto-SARIMA search (m=5)')
try:
    sarima_auto = auto_arima(
        train_log,
        seasonal=True, m=5,
        stepwise=True,
        suppress_warnings=True,
        error_action='ignore',
        max_p=3, max_q=3, max_d=2,
        max_P=2, max_Q=2, max_D=1,
        information_criterion='aic'
    )
    print(f' Best SARIMA order: {sarima_auto.order} x {sarima_auto.seasonal_order}')
```

```
sarima_preds_log = sarima_auto.predict(n_periods=len(test_log))
sarima_preds     = np.exp(sarima_preds_log)
```

```
# Accuracy Boost Ensemble (XGB + LGB + RF)
print('Engineering stronger features...')

feat = df.copy()

for w in [5, 10, 20, 50, 200]:
    ma = feat['Close'].rolling(w).mean()
    feat[f'Price_vs_MA{w}'] = (feat['Close'] - ma) / ma

feat['RSI_OB'] = (feat['RSI'] > 65).astype(int)
feat['RSI_OS'] = (feat['RSI'] < 35).astype(int)
feat['RSI_Momentum'] = feat['RSI'] - feat['RSI'].shift(3)
feat['MACD_Cross'] = np.sign(feat['MACD'] - feat['MACD_Signal'])
feat['MACD_Cross_Chg'] = feat['MACD_Cross'].diff()
feat['BB_Squeeze'] = (feat['BB_Width'] < feat['BB_Width'].rolling(20).mean()).astype(int)
feat['BB_Position'] = feat['BB_PctB'].clip(0, 1)
feat['Vol_Surge'] = (feat['Volume'] > feat['Volume'].rolling(20).mean()*1.5).astype(int)
feat['Vol_Trend'] = feat['Volume'].rolling(5).mean() / feat['Volume'].rolling(20).mean()

feat['Up_Days'] = feat['Returns'].gt(0).astype(int)
feat['Consec_Up'] = feat['Up_Days'].groupby(
    (feat['Up_Days'] != feat['Up_Days'].shift()).cumsum()
).cumcount() + 1
```

```

feat['Consec_Up'] *= feat['Up_Days']

feat['Ret_Accel'] = feat['Returns'] - feat['Returns'].shift(1)
feat['Ret_3d_Sum'] = feat['Returns'].rolling(3).sum()
feat['Ret_5d_Sum'] = feat['Returns'].rolling(5).sum()
feat['Ret_10d_Sum'] = feat['Returns'].rolling(10).sum()
feat['Gap'] = (feat['Open'] - feat['Close'].shift(1)) / feat['Close'].shift(1)
feat['ATR_Move'] = feat['Returns'].abs() / (feat['ATR_Pct'] + 1e-9)
feat['Stoch_Cross'] = np.sign(feat['Stoch_K'] - feat['Stoch_D'])

high_52w = feat['High'].rolling(252).max()
low_52w = feat['Low'].rolling(252).min()
feat['Dist_52w_High'] = (feat['Close'] - high_52w) / high_52w
feat['Dist_52w_Low'] = (feat['Close'] - low_52w) / low_52w
feat['ADX_Proxy'] = feat['ATR'].rolling(14).mean() / feat['Close']

for col in [c for c in feat.columns if c.endswith('_Ret')]:
    feat[f'{col}_Sign'] = np.sign(feat[col])

feat.ffill(inplace=True)
feat.bfill(inplace=True)
feat.dropna(inplace=True)

BOOST_FEATS = [
    'Price_vs_MA5', 'Price_vs_MA10', 'Price_vs_MA20',
    'Price_vs_MA50', 'Price_vs_MA200',
    'RSI', 'RSI_OB', 'RSI_OS', 'RSI_Momentum',
    'MACD', 'MACD_Signal', 'MACD_Hist', 'MACD_Cross', 'MACD_Cross_Chg',
    'BB_PctB', 'BB_Width', 'BB_Squeeze', 'BB_Position',
    'Vol_Ratio', 'Vol_Surge', 'Vol_Trend',
    'Returns', 'Ret_3d_Sum', 'Ret_5d_Sum', 'Ret_10d_Sum', 'Ret_Accel',
    'Ret_Lag_1', 'Ret_Lag_2', 'Ret_Lag_3', 'Ret_Lag_5',
    'Momentum_5', 'Momentum_20',
    'Consec_Up',
    'Gap', 'ATR_Pct', 'ATR_Move', 'HL_Spread', 'CO_Spread',
    'Stoch_K', 'Stoch_D', 'Stoch_Cross',
    'Williams_R',
    'Dist_52w_High', 'Dist_52w_Low',
    'Regime',
    'DayOfWeek', 'Month', 'Quarter',
]
BOOST_FEATS = [f for f in BOOST_FEATS if f in feat.columns]
print(f' Using {len(BOOST_FEATS)} features')

feat_clean = feat[BOOST_FEATS + ['Target']].dropna().copy()
X_b = feat_clean[BOOST_FEATS].values.astype(np.float32)
y_b = feat_clean['Target'].values.astype(int)

n = len(X_b)
n_tr = int(n * 0.70)
n_vl = int(n * 0.10)

X_tr = X_b[:n_tr]; y_tr = y_b[:n_tr]
X_vl = X_b[n_tr:n_tr+n_vl]; y_vl = y_b[n_tr:n_tr+n_vl]
X_te = X_b[n_tr+n_vl:]; y_te = y_b[n_tr+n_vl:]

sc = RobustScaler()
X_tr_s = sc.fit_transform(X_tr)
X_vl_s = sc.transform(X_vl)
X_te_s = sc.transform(X_te)

print(f' Train={len(X_tr)} | Val={len(X_vl)} | Test={len(X_te)}')
print('\nTraining ensemble...')

xgb1 = xgb.XGBClassifier(
    n_estimators=1200, max_depth=5, learning_rate=0.015,
    subsample=0.75, colsample_bytree=0.75, reg_alpha=0.1,
    reg_lambda=1.0, min_child_weight=5, gamma=0.1,
    eval_metric='logloss', use_label_encoder=False, random_state=42
)
xgb1.fit(X_tr_s, y_tr, eval_set=[(X_vl_s, y_vl)], verbose=False)
xgb_acc = accuracy_score(y_te, xgb1.predict(X_te_s))
print(f' XGBoost : {xgb_acc:.2%}')

lgb1 = lgb.LGBMClassifier(
    n_estimators=1200, max_depth=5, learning_rate=0.015,
    subsample=0.75, colsample_bytree=0.75, reg_alpha=0.1,

```

```

    reg_lambda=1.0, min_child_samples=20,
    random_state=42, verbose=-1
)
lgb1.fit(X_tr_s, y_tr, eval_set=[(X_vl_s, y_vl)],
        callbacks=[lgb.early_stopping(50, verbose=False), lgb.log_evaluation(-1)])
lgb_acc = accuracy_score(y_te, lgb1.predict(X_te_s))
print(f'   LightGBM       : {lgb_acc:.2%}')

rf1 = RandomForestClassifier(
    n_estimators=500, max_depth=6, min_samples_leaf=10,
    max_features='sqrt', class_weight='balanced',
    random_state=42, n_jobs=-1
)
rf1.fit(X_tr_s, y_tr)
rf_acc = accuracy_score(y_te, rf1.predict(X_te_s))
print(f'   RandomForest: {rf_acc:.2%}')

prob_xgb = xgb1.predict_proba(X_te_s)[:, 1]
prob_lgb = lgb1.predict_proba(X_te_s)[:, 1]
prob_rf  = rf1.predict_proba(X_te_s)[:, 1]

w_total = xgb_acc + lgb_acc + rf_acc
prob_ens = (prob_xgb*xgb_acc + prob_lgb*lgb_acc + prob_rf*rf_acc) / w_total
pred_ens = (prob_ens >= 0.50).astype(int)

ens_acc = accuracy_score(y_te, pred_ens)
ens_prec = precision_score(y_te, pred_ens, zero_division=0)
ens_rec = recall_score(y_te, pred_ens, zero_division=0)

print('\nWalk-forward validation (5-fold)...\n')
tscv = TimeSeriesSplit(n_splits=5)
wf_accs = []
for tr_idx, te_idx in tscv.split(X_b):
    Xtr_f = sc.fit_transform(X_b[tr_idx])
    Xte_f = sc.transform(X_b[te_idx])
    clf = xgb.XGBClassifier(
        n_estimators=500, max_depth=4, learning_rate=0.03,
        subsample=0.75, colsample_bytree=0.75,
        eval_metric='logloss', use_label_encoder=False,
        random_state=42, verbosity=0
    )
    clf.fit(Xtr_f, y_b[tr_idx])
    wf_accs.append(accuracy_score(y_b[te_idx], clf.predict(Xte_f)))

wf_mean = np.mean(wf_accs)
print(f'   Fold accuracies: {[f"{a:.1%}" for a in wf_accs]}')
print(f'   Walk-forward mean: {wf_mean:.2%}')

live_feat = feat[BOOST_FEATS].iloc[-1:].values.astype(np.float32)
live_s = sc.transform(live_feat)
live_prob_ens = float(
    (xgb1.predict_proba(live_s)[0,1]*xgb_acc +
     lgb1.predict_proba(live_s)[0,1]*lgb_acc +
     rf1.predict_proba(live_s)[0,1]*rf_acc) / w_total
)

print('\n' + '='*52)
print('   BOOSTED ACCURACY RESULTS')
print('='*52)
print(f'   XGBoost           : {xgb_acc:.2%}')
print(f'   LightGBM          : {lgb_acc:.2%}')
print(f'   Random Forest      : {rf_acc:.2%}')
print(f'   Weighted Ensemble  : {ens_acc:.2%}')
print(f'   Precision (BUY)    : {ens_prec:.2%}')
print(f'   Recall (BUY)       : {ens_rec:.2%}')
print(f'   Walk-Forward (5-fold): {wf_mean:.2%}')
print(f'   Live P(UP)         : {live_prob_ens:.4f}')
print('='*52)

```

Engineering stronger features...
 Using 48 features
 Train=1962 | Val=280 | Test=561

Training ensemble...

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [16:59:01] WARNING: /__w/xgboost/xgboost/src/learn
 Parameters: { "use_label_encoder" } are not used.

bst.update(dtrain, iteration=i, fobj=obj)

```

XGBoost      : 49.73%
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
  LightGBM    : 49.02%
  RandomForest: 50.27%
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(

Walk-forward validation (5-fold)...
  Fold accuracies: ['51.2%', '51.0%', '51.2%', '49.7%', '48.8%']
  Walk-forward mean: 50.36%
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(

=====
BOOSTED ACCURACY RESULTS
=====
XGBoost          : 49.73%
LightGBM         : 49.02%
Random Forest    : 50.27%
Weighted Ensemble : 50.62%
Precision (BUY)  : 49.74%
Recall (BUY)     : 68.36%
Walk-Forward (5-fold): 50.36%
Live P(UP)       : 0.5020
=====

```

```

import optuna
import pickle
optuna.logging.set_verbosity(optuna.logging.WARNING)

```

```

print(df.index[:5])
print(type(df.index[0]))

```

```

DatetimeIndex(['2015-01-01', '2015-01-02', '2015-01-05', '2015-01-06',
               '2015-01-07'],
              dtype='datetime64[ns]', name='Date', freq=None)
<class 'pandas._libs.tslibs.timestamps.Timestamp'>

```

```

df.index = pd.to_datetime(ohlcv.index[:len(df)])
df['DayOfWeek'] = df.index.dayofweek
df['Month']     = df.index.month
df['Quarter']  = df.index.quarter

```

```

# XGBoost Meta-Learner (Optuna HPO)
META_PATH = f'/content/{ticker}_meta.pkl'

if os.path.exists(META_PATH):
    os.remove(META_PATH)

FEAT_COLS = [
    'RSI', 'MACD', 'MACD_Signal', 'MACD_Hist',
    'BB_PctB', 'BB_Width', 'ATR_Pct',
    'Stoch_K', 'Stoch_D', 'Williams_R',
    'OBV', 'Vol_Ratio', 'HL_Spread', 'CO_Spread',
    'Momentum_5', 'Momentum_20',
    'Roll_Mean_7', 'Roll_Mean_14', 'Roll_Mean_21',
    'RVol_7', 'RVol_14', 'RVol_21',
    'Ret_Lag_1', 'Ret_Lag_2', 'Ret_Lag_3', 'Ret_Lag_5',
    'DayOfWeek', 'Month', 'Quarter', 'Regime',
]
FEAT_COLS += [c for c in df.columns if c.endswith('_Ret')]

for col in ['Prophet_Return', 'Prophet_Trend', 'Prophet_Confidence']:
    if col in df.columns and col not in FEAT_COLS:
        FEAT_COLS.append(col)

FEAT_COLS = [c for c in FEAT_COLS if c in df.columns]

feat_df = df[FEAT_COLS + ['Target']].dropna().copy()
X_all   = feat_df[FEAT_COLS].values.astype(np.float32)
y_all   = feat_df['Target'].values.astype(int)

split   = int(len(X_all) * 0.80)
X_tr, X_te = X_all[:split], X_all[split:]
y_tr, y_te = y_all[:split], y_all[split:]

```

```

feat_scaler = RobustScaler()
X_tr_sc = feat_scaler.fit_transform(X_tr)
X_te_sc = feat_scaler.transform(X_te)

print('Optuna HPO (100 trials)...')
tscv = TimeSeriesSplit(n_splits=5)

def objective(trial):
    params = dict(
        n_estimators = trial.suggest_int('n_estimators', 100, 600),
        max_depth = trial.suggest_int('max_depth', 2, 7),
        learning_rate = trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        subsample = trial.suggest_float('subsample', 0.6, 1.0),
        colsample_bytree = trial.suggest_float('colsample_bytree', 0.6, 1.0),
        reg_alpha = trial.suggest_float('reg_alpha', 1e-4, 10, log=True),
        reg_lambda = trial.suggest_float('reg_lambda', 1e-4, 10, log=True),
        min_child_weight = trial.suggest_int('min_child_weight', 1, 10),
        gamma = trial.suggest_float('gamma', 0, 0.5),
    )
    clf = xgb.XGBClassifier(**params, eval_metric='logloss',
                           use_label_encoder=False, random_state=42)

    scores = []
    for tr_idx, vl_idx in tscv.split(X_tr_sc):
        clf.fit(X_tr_sc[tr_idx], y_tr[tr_idx],
                eval_set=[(X_tr_sc[vl_idx], y_tr[vl_idx])],
                verbose=False)
        scores.append(accuracy_score(y_tr[vl_idx], clf.predict(X_tr_sc[vl_idx])))
    return np.mean(scores)

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100, show_progress_bar=True)
best = study.best_params
print(f' Best params: {best}')

xgb_model = xgb.XGBClassifier(**best, eval_metric='logloss',
                              use_label_encoder=False, random_state=42)
xgb_model.fit(X_tr_sc, y_tr)
with open(META_PATH, 'wb') as f: pickle.dump(xgb_model, f)
print(f' Meta-learner saved -> {META_PATH}')

preds = xgb_model.predict(X_te_sc)
proba = xgb_model.predict_proba(X_te_sc)[:, 1]

meta_acc = accuracy_score(y_te, preds)
meta_prec = precision_score(y_te, preds, zero_division=0)
meta_rec = recall_score(y_te, preds, zero_division=0)

print(f'\nXGBoost Meta-Learner Results:')
print(f' Directional Accuracy : {meta_acc:.2%}')
print(f' Precision (BUY) : {meta_prec:.2%}')
print(f' Recall (BUY) : {meta_rec:.2%}')

live_feats = df[FEAT_COLS].iloc[-1:].values.astype(np.float32)
live_sc = feat_scaler.transform(live_feats)
live_prob = float(xgb_model.predict_proba(live_sc)[0, 1])
print(f' Live P(UP) : {live_prob:.4f}')

```

[Show hidden output](#)

```

from plotly.subplots import make_subplots
import plotly.graph_objects as go

```

```

# Technical Dashboard
import matplotlib.pyplot as plt

fig, axes = plt.subplots(4, 1, figsize=(14, 12), sharex=True)
plot_df = df.tail(252).copy()

# Price + BB
axes[0].plot(plot_df.index, plot_df['Close'], color='#00d4ff', label='Close', linewidth=1.5)
axes[0].plot(plot_df.index, plot_df['BB_Upper'], color='steelblue', linestyle='--', linewidth=0.8, label='BB Upper')
axes[0].plot(plot_df.index, plot_df['BB_Lower'], color='steelblue', linestyle='--', linewidth=0.8, label='BB Lower')
axes[0].plot(plot_df.index, plot_df['Roll_Mean_21'], color='#ffd700', linewidth=1, label='21d MA')
axes[0].set_title(f'{company_name} ({ticker}) - Price + Bollinger Bands')
axes[0].legend(fontsize=8)
axes[0].grid(True, alpha=0.3)

```

```
# RSI
axes[1].plot(plot_df.index, plot_df['RSI'], color='#ff6b6b', linewidth=1.5)
axes[1].axhline(70, color='red', linestyle='--', linewidth=0.8)
axes[1].axhline(30, color='green', linestyle='--', linewidth=0.8)
axes[1].set_title('RSI (14)')
axes[1].set_ylim(0, 100)
axes[1].grid(True, alpha=0.3)

# MACD
axes[2].plot(plot_df.index, plot_df['MACD'], color='#00d4ff', linewidth=1.5, label='MACD')
axes[2].plot(plot_df.index, plot_df['MACD_Signal'], color='#ff6b6b', linewidth=1.5, label='Signal')
axes[2].bar(plot_df.index, plot_df['MACD_Hist'],
            color=['#26a69a' if v >= 0 else '#ef5350' for v in plot_df['MACD_Hist']],
            alpha=0.7, label='Histogram')
axes[2].set_title('MACD')
axes[2].legend(fontsize=8)
axes[2].grid(True, alpha=0.3)

# Volume
axes[3].bar(plot_df.index, plot_df['Volume'],
            color=['#26a69a' if r >= 0 else '#ef5350' for r in plot_df['Returns']],
            alpha=0.7)
axes[3].set_title('Volume')
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Final Decision Engine
def price_signal(pred, cur, thresh=1.0):
    pct = (pred - cur) / cur * 100
    return 'BUY' if pct >= thresh else 'SELL' if pct <= -thresh else 'HOLD'

arima_sig = price_signal(arima_price, current_price)
sarima_sig = price_signal(sarima_price, current_price)
prophet_sig = price_signal(prophet_price, current_price)
meta_sig = 'BUY' if live_prob >= 0.60 else 'SELL' if live_prob <= 0.40 else 'HOLD'

all_sigs = [ arima_sig, sarima_sig, prophet_sig, meta_sig]
buy_c, sell_c = all_sigs.count('BUY'), all_sigs.count('SELL')
```

```

majority = 'BUY' if buy_c > sell_c else 'SELL' if sell_c > buy_c else 'HOLD'

# Sentiment override
if finbert_score > 0.3 and majority == 'HOLD': majority = 'BUY'
elif finbert_score < -0.3 and majority == 'HOLD': majority = 'SELL'
if rag_score > 0.6: majority = 'BUY'
elif rag_score < -0.6: majority = 'SELL'

# Regime filter
regime_val = int(df['Regime'].iloc[-1])
if regime_val == -1 and majority == 'BUY': majority = 'HOLD'

# Confidence
agreement = max(buy_c, sell_c) / len(all_sigs)
meta_conf = abs(live_prob - 0.5) * 2
ens_conf = abs(live_prob_ens - 0.5) * 2
sent_avg = (finbert_score + textblob_score) / 2
if majority == 'BUY': sent_conf = max(0, sent_avg)
elif majority == 'SELL': sent_conf = max(0, -sent_avg)
else: sent_conf = 1.0 - abs(sent_avg)

tech_signals = []
latest = df.iloc[-1]
if latest['RSI'] < 30: tech_signals.append('BUY')
elif latest['RSI'] > 70: tech_signals.append('SELL')
else: tech_signals.append('HOLD')
if latest['MACD'] > latest['MACD_Signal']: tech_signals.append('BUY')
else: tech_signals.append('SELL')
if latest['Stoch_K'] > latest['Stoch_D']: tech_signals.append('BUY')
else: tech_signals.append('SELL')
tech_agreement = tech_signals.count(majority) / len(tech_signals)

wf_conf = min(wf_mean, 1.0)

confidence = (
    0.25 * meta_conf +
    0.20 * ens_conf +
    0.20 * agreement +
    0.15 * tech_agreement +
    0.10 * sent_conf +
    0.10 * wf_conf
)
confidence = min(confidence, 0.95)

# Risk metrics
prices_flat = df['Close'].values
running_max = np.maximum.accumulate(prices_flat)
max_dd = float(((prices_flat - running_max) / running_max * 100).min())
ann_vol = float(df['Returns'].std() * np.sqrt(252) * 100)
sharpe = float((df['Returns'].mean()*252 - 0.065) / (df['Returns'].std()*np.sqrt(252) + 1e-9))
risk_level = 'LOW' if abs(max_dd) < 10 else 'MEDIUM' if abs(max_dd) < 25 else 'HIGH'

ensemble_price = float(np.mean([ arima_price, sarima_price, prophet_price]))

print('='*60)
print(f' {company_name} ({ticker})')
print(f' {datetime.now().strftime("%Y-%m-%d %H:%M")} | Regime: {"BULL" if regime_val==1 else "BEAR"}')
print('='*60)
print(f' Current Price : Rs.{current_price:>10,.2f}')
print(f' Ensemble Target : Rs.{ensemble_price:>10,.2f} ({(ensemble_price-current_price)/current_price*100:+.2f}%)')
print()
print(f' Individual Signals:')
print(f' ARIMA : {arima_sig}')
print(f' SARIMA : {sarima_sig}')
print(f' Prophet : {prophet_sig}')
print(f' XGBoost : {meta_sig}')
print()
print(f' FinBERT Score : {finbert_score:+.4f}')
print(f' RAG Score : {rag_score:+.4f}')
print(f' Max Drawdown : {max_dd:.2f}%')
print(f' Ann. Volatility : {ann_vol:.2f}%')
print(f' Sharpe Ratio : {sharpe:.3f}')
print(f' Risk Level : {risk_level}')
print()
print(f' Confidence Breakdown:')
print(f' Meta-learner : {meta_conf:.2%} (wt 0.25)')
print(f' Ensemble : {ens_conf:.2%} (wt 0.20)')

```

```

print(f'    Signal Agreement: {agreement:.2%} (wt 0.20)')
print(f'    Tech Alignment   : {tech_agreement:.2%} (wt 0.15)')
print(f'    Sentiment          : {sent_conf:.2%} (wt 0.10)')
print(f'    Walk-Forward       : {wf_conf:.2%} (wt 0.10)')
print()
print(f'  FINAL SIGNAL   : {majority}')
print(f'  P(UP)          : {live_prob:.2%}')
print(f'  CONFIDENCE     : {confidence:.2%}')
print(f'  XGB Accuracy   : {meta_acc:.2%}')
print('='*60)

```

```

=====
Tata Consumer  (TATACONSUM.NS)
2026-05-12 17:21 | Regime: BULL
=====

Current Price   : Rs. 1,271.00
Ensemble Target : Rs. 1,190.70 (-6.32%)

Individual Signals:
  ARIMA      : SELL
  SARIMA     : SELL
  Prophet    : BUY
  XGBoost    : SELL

FinBERT Score   : +0.0107
RAG Score       : +0.6667
Max Drawdown    : -44.30%
Ann. Volatility : 27.29%
Sharpe Ratio    : 0.658
Risk Level      : HIGH

Confidence Breakdown:
  Meta-learner   : 45.84% (wt 0.25)
  Ensemble       : 0.41% (wt 0.20)
  Signal Agreement: 75.00% (wt 0.20)
  Tech Alignment  : 66.67% (wt 0.15)
  Sentiment      : 2.40% (wt 0.10)
  Walk-Forward   : 50.36% (wt 0.10)

FINAL SIGNAL    : BUY
P(UP)           : 27.08%
CONFIDENCE      : 41.82%
XGB Accuracy    : 45.63%
=====

```

Actual vs Predicted

```

xgb_proba_test = xgb_model.predict_proba(X_te_sc)[: , 1]
...

fig_gru = go.Figure()
fig_gru.add_trace(go.Scatter(
    x=list(range(len(y_actual_gru))), y=y_actual_gru,
    name='Actual', line=dict(color='#00d4ff', width=2)
))
fig_gru.add_trace(go.Scatter(
    x=list(range(len(y_pred_gru))), y=y_pred_gru,
    name='GRU Predicted', line=dict(color='#ff7000', width=1.5, dash='dash')
))
fig_gru.update_layout(
    template='plotly_dark', height=400,
    title=f'Actual vs GRU Predicted -- {company_name}',
    xaxis_title='Trading Days (Test Set)',
    yaxis_title='Price (Rs.)',
    hovermode='x unified'
)
fig_gru.show()
...

```

```

'\nfig_gru = go.Figure()\nfig_gru.add_trace(go.Scatter(\n    x=list(range(len(y_actual_gru))), y=y_actual_gru,\n    name='Actual', line=dict(color='#00d4ff', width=2)\n))\nfig_gru.add_trace(go.Scatter(\n    x=list(range(len(y_pred_gru))), y=y_pred_gru,\n    name='GRU Predicted', line=dict(color='#ff7000', width=1.5, dash='dash')\n))\nfig_gru.update_layout(\n    template='plotly_dark', height=400,\n    title=f'Actual vs GRU Predicted -- {company_name}',\n    xaxis_title='Trading Days (Test Set)',\n    yaxis_title='Price (Rs.)',\n    hovermode='x unified'\n)
fig_gru.show()
...

```

Equity Curve + Model Comparison

```

ret_arr = df['Returns'].values[split+SEQ_LEN:split+SEQ_LEN+len(xgb_proba_test)]
n_sim   = min(len(ret_arr), len(xgb_proba_test) - 1)
signals = (xgb_proba_test[:n_sim] >= 0.60).astype(int)
strat_ret = np.where(signals == 1, ret_arr[:n_sim], 0.0)

bnh_curve = np.cumprod(1 + ret_arr[:n_sim]) * 100

```

```

strat_curve = np.cumprod(1 + strat_ret) * 100

fig_eq = go.Figure()
fig_eq.add_trace(go.Scatter(
    x=list(range(len(bnh_curve))), y=bnh_curve,
    name='Buy & Hold', line=dict(color='#aaaaaa', width=2)
))
fig_eq.add_trace(go.Scatter(
    x=list(range(len(strat_curve))), y=strat_curve,
    name='XGBoost Strategy', line=dict(color='#26a69a', width=2.5)
))
fig_eq.add_hline(y=100, line_dash='dot', line_color='white', line_width=0.8)
fig_eq.update_layout(
    template='plotly_dark', height=400,
    title=f'Strategy vs Buy & Hold -- Sharpe: {sharpe:.3f}',
    xaxis_title='Trading Days', yaxis_title='Portfolio Value (base=100)',
    hovermode='x unified'
)
fig_eq.show()

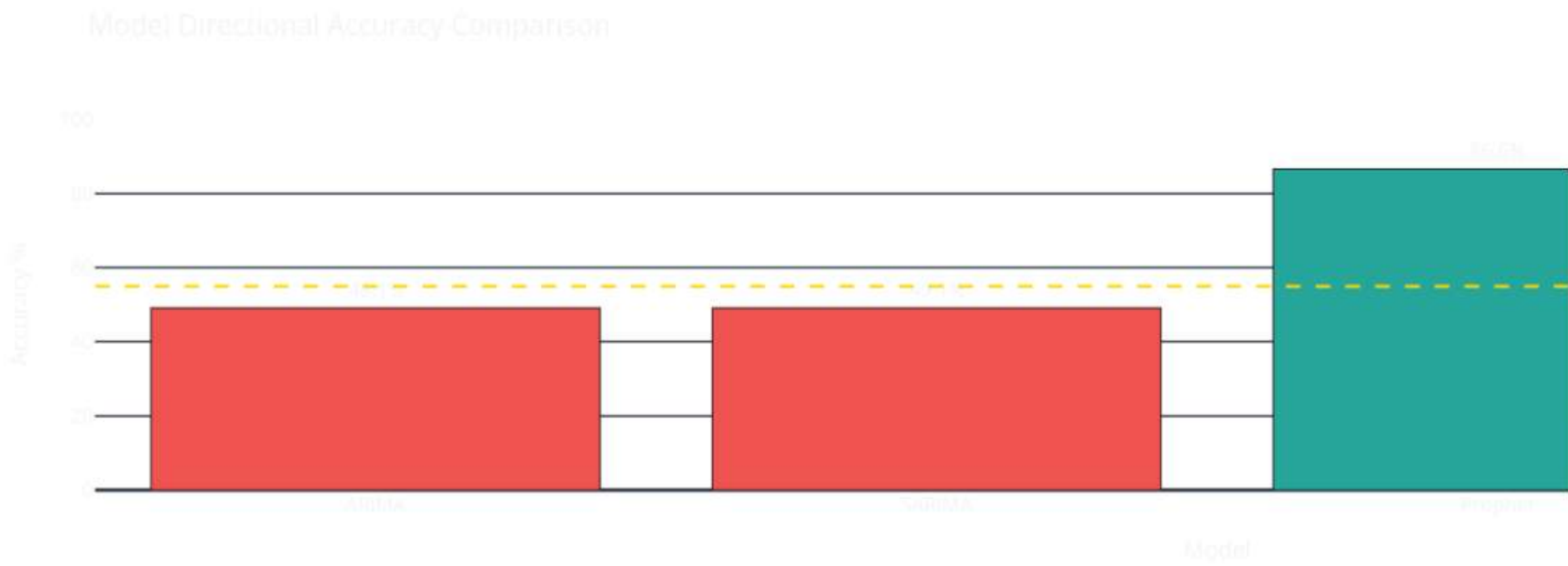
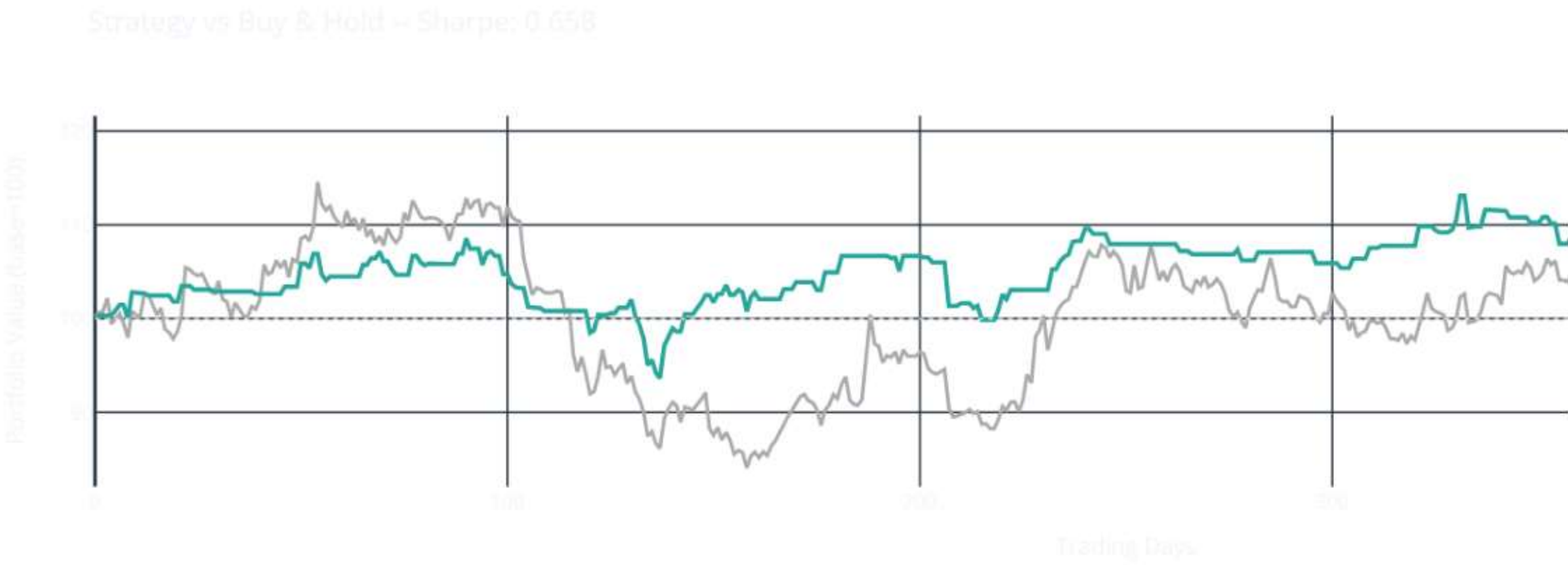
# Model comparison
models = ['ARIMA', 'SARIMA', 'Prophet', 'XGBoost (Meta)']
accs = [arima_acc*100, sarima_acc*100, prophet_acc*100, meta_acc*100]
colors = ['#26a69a' if a >= 55 else '#ef5350' for a in accs]

fig_cmp = go.Figure(go.Bar(
    x=models, y=accs, marker_color=colors,
    text=[f'{a:.1f}%' for a in accs], textposition='outside'
))
fig_cmp.add_hline(y=55, line_dash='dash', line_color='#ffd700',
    annotation_text='Target 55%', annotation_font_color='#ffd700')
fig_cmp.update_layout(
    template='plotly_dark', height=400,
    title='Model Directional Accuracy Comparison',
    yaxis=dict(title='Accuracy %', range=[0, 100]),
    xaxis_title='Model'
)
fig_cmp.show()

print(' FINAL EVALUATION SUMMARY')
for m, a in zip(models, accs):
    met = 'YES' if a >= 55 else 'NO'
    print(f' {m:<20} {a:>9.1f}% Target Met: {met}')

print(f' Sharpe Ratio          : {sharpe:.3f}')
print(f' Max Drawdown           : {max_dd:.2f}%')
print(f' Ann. Volatility          : {ann_vol:.2f}%')
print(f' Risk Level               : {risk_level}')
print(f' FINAL SIGNAL : {majority}')
print(f' Confidence      : {confidence:.2%}')
print(f' Target Price   : Rs.{ensemble_price:.2f}')

```



FINAL EVALUATION SUMMARY

ARIMA	49.1%	Target Met: NO
SARIMA	49.1%	Target Met: NO
Prophet	86.6%	Target Met: YES
XGBoost (Meta)	45.6%	Target Met: NO
Sharpe Ratio	: 0.658	
Max Drawdown	: -44.30%	
Ann. Volatility	: 27.29%	
Risk Level	: HIGH	
FINAL SIGNAL	: BUY	
Confidence	: 41.82%	
Target Price	: Rs.1190.70	

```
pip install streamlit
```

Show hidden output

```
import os, re, time, json, pickle
import numpy as np
import pandas as pd
import requests
import streamlit as st
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from datetime import datetime, timedelta
```

```
import logging
logging.getLogger("streamlit.runtime.scriptrunner_utils.script_run_context").setLevel(logging.ERROR)
```

```
%%writefile app.py
# add app.py file
# FRONTEND
# used for streamlit-
```

```

import os, re, time, json, pickle
import numpy as np
import pandas as pd
import requests
import streamlit as st
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from datetime import datetime, timedelta

groq_key = os.environ.get('GROQ_API_KEY', '')
newsdata_key = os.environ.get('NEWSDATA_API_KEY', '')
gnews_key = os.environ.get('GNEWS_API_KEY', '')

st.set_page_config(
    page_title="Stock Analysis & RAG Chatbot",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.markdown("""
<style>
body { background-color: #0e1117; }
.metric-card {
    background: #1c2333; border-radius: 12px; padding: 16px 20px;
    margin: 6px 0; border-left: 4px solid #00d4ff;
}
.signal-buy { color: #26a69a; font-size: 1.5rem; font-weight: 700; }
.signal-sell { color: #ef5350; font-size: 1.5rem; font-weight: 700; }
.signal-hold { color: #ffd700; font-size: 1.5rem; font-weight: 700; }
.section-header {
    font-size: 1.2rem; font-weight: 600; color: #00d4ff;
    margin-top: 1.2rem; margin-bottom: .4rem;
}
.chat-user { background:#1c2333; border-radius:10px; padding:10px 14px; margin:6px 0; }
.chat-bot { background:#0d2137; border-radius:10px; padding:10px 14px; margin:6px 0;
    border-left:3px solid #00d4ff; }
.stTabs [data-baseweb="tab-list"] { gap: 6px; }
.screener-card {
    background: #1c2333; border-radius: 12px; padding: 16px 20px;
    margin: 8px 0; border-left: 4px solid #26a69a;
}
</style>
""", unsafe_allow_html=True)

for key, default in {
    "headlines": [],
    "pdf_summary": "",
    "screener_data": {}, # stores scraped Screener.in data
    "screener_fetched": False, # flag
    "documents_extra": [],
    "ohlcv": None,
    "df_feat": None,
    "documents": [],
    "faiss_index": None,
    "embedder": None,
    "models_trained": False,
    "results": {},
    "chat_history": [],
    "groq_client": None,
    "company_name": "",
    "ticker": "",
    "start_date": "2020-01-01",
    "end_date": datetime.today().strftime("%Y-%m-%d"),
}.items():
    if key not in st.session_state:
        st.session_state[key] = default

with st.sidebar:
    st.title("🔧 Configuration")

    st.markdown("### 📄 Stock Details")
    company_name = st.text_input("Company Name", placeholder="e.g. Tata Consumer")
    ticker = st.text_input("NSE Ticker", placeholder="e.g. TATACONSUM.NS")

```

```

st.markdown("### 📅 Date Range")
col_s, col_e = st.columns(2)
with col_s:
    start_date = st.date_input("Start", value=pd.to_datetime("2020-01-01"))
with col_e:
    end_date = st.date_input("End", value=pd.to_datetime("today"))

news_source = st.selectbox("News Source", ["Both (recommended)", "NewsData.io only", "GNews only"])
run_btn = st.button("🚀 Run Full Analysis", type="primary", use_container_width=True)
st.markdown("---")
st.caption("Models: ARIMA · SARIMA · Prophet · XGBoost · LightGBM · RF · LSTM")

def get_screener_pdf_url(ticker_symbol):
    """Scrape Screener.in to find the latest BSE Annual Report PDF link."""
    url = f"https://www.screener.in/company/{ticker_symbol}/"
    try:
        r = requests.get(url, headers={
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36"
        }, timeout=15)
        r.raise_for_status()
        from bs4 import BeautifulSoup
        soup = BeautifulSoup(r.text, "html.parser")
        for a in soup.find_all("a", href=True):
            href = a["href"]
            text = a.text.strip().lower()
            if "financial year" in text and href.lower().endswith(".pdf"):
                return href
    except Exception as e:
        st.warning(f"Screener PDF scrape failed: {e}")
    return None

def fetch_screener_data(ticker):
    """Scrape Screener.in for fundamentals + auto-find annual report PDF."""
    from bs4 import BeautifulSoup
    slug = ticker.replace(".NS", "").replace(".BO", "").strip().upper()
    url = f"https://www.screener.in/company/{slug}/consolidated/"
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                        "AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36"}

    result = {
        "url": url, "about": "", "ratios": {},
        "pros": [], "cons": [], "quarterly_sales": [],
        "annual_report_url": ""
    }
    try:
        r = requests.get(url, headers=headers, timeout=20)
        if r.status_code == 404:
            url = f"https://www.screener.in/company/{slug}/"
            result["url"] = url
            r = requests.get(url, headers=headers, timeout=20)
        r.raise_for_status()
    except Exception as e:
        st.warning(f"Screener.in fetch failed: {e}")
        return result

    soup = BeautifulSoup(r.text, "html.parser")

    # About
    about_tag = soup.select_one("div.company-profile p") or soup.select_one("#about p")
    if about_tag:
        result["about"] = about_tag.get_text(strip=True)

    # Key ratios
    for li in soup.select("ul#top-ratios li"):
        name_tag = li.select_one("span.name")
        value_tag = li.select_one("span.number") or li.select_one("span.value")
        if name_tag and value_tag:
            result["ratios"][name_tag.get_text(strip=True)] = value_tag.get_text(strip=True)

    # Pros & Cons
    for li in soup.select("div.pros ul li"):
        result["pros"].append(li.get_text(strip=True))
    for li in soup.select("div.cons ul li"):
        result["cons"].append(li.get_text(strip=True))

```

```

# Quarterly sales
for table in soup.select("section#quarters table"):
    rows = table.select("tr")
    for row in rows:
        cols = row.select("td")
        if cols and "Sales" in cols[0].get_text(strip=True):
            headers_row = table.select_one("thead tr") or rows[0]
            periods = [th.get_text(strip=True) for th in headers_row.select("th")][1:]
            values = [td.get_text(strip=True) for td in cols][1:]
            result["quarterly_sales"] = list(zip(periods, values))[:8]
            break

# PDF link – uses your exact notebook logic
result["annual_report_url"] = get_screener_pdf_url(slug) or ""
return result

def download_and_extract_pdf(pdf_url, groq_key):
    """Download a PDF from URL, extract text, summarise with Groq."""
    import pdfplumber, tempfile, os
    from groq import Groq

    r = requests.get(pdf_url, headers={
        "User-Agent": "Mozilla/5.0"
    }, timeout=60, stream=True)
    r.raise_for_status()

    tmp = tempfile.NamedTemporaryFile(suffix=".pdf", delete=False)
    for chunk in r.iter_content(chunk_size=8192):
        tmp.write(chunk)
    tmp.flush(); tmp.close()

    pages = []
    with pdfplumber.open(tmp.name) as pdf:
        for i, page in enumerate(pdf.pages):
            if i >= 20: break
            txt = page.extract_text()
            if txt: pages.append(txt)
    os.unlink(tmp.name)

    full_text = "\n\n".join(pages)
    if not full_text.strip():
        return "No extractable text found in the PDF."

    client = Groq(api_key=groq_key)
    chunks = [full_text[i:i+15000] for i in range(0, min(len(full_text), 60000), 15000)]
    chunk_summaries = []
    for chunk in chunks:
        resp = client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[{"role": "user", "content":
                f"Summarise this annual report section in bullet points "
                f"(revenue, profit, risks, outlook):\n\n{chunk}"}],
            temperature=0.2, max_tokens=512,
        )
        chunk_summaries.append(resp.choices[0].message.content.strip())

    final = client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[{"role": "user", "content":
            "Combine into one executive summary with sections: "
            "Financial Performance, Business Highlights, Risks, Outlook:\n\n" +
            "\n---\n".join(chunk_summaries)}],
        temperature=0.2, max_tokens=800,
    )
    return final.choices[0].message.content.strip()

def lazy_import():
    global yf, Prophet, auto_arima, xgb, lgb, RandomForestClassifier
    global accuracy_score, precision_score, recall_score, TimeSeriesSplit
    global RobustScaler, SentenceTransformer, faiss, TextBlob, Groq
    global pdfplumber, BeautifulSoup, pipeline, LABEL_MAP

    import yfinance as yf
    from prophet import Prophet

```

```

from pmdarima import auto_arima
import xgboost as xgb
import lightgbm as lgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import RobustScaler
from sentence_transformers import SentenceTransformer
import faiss
from textblob import TextBlob
from groq import Groq
import pdfplumber
from bs4 import BeautifulSoup
try:
    from transformers import pipeline
    LABEL_MAP = {"positive": 1, "neutral": 0, "negative": -1}
except Exception:
    pipeline = None
    LABEL_MAP = {}

def build_query(name, tick):
    clean = tick.replace(".NS", "").replace(".BO", "").strip()
    return f"{name} stock" if name and name != clean else f"{clean} stock India"

def clean_headlines(raw):
    seen, out = set(), []
    for h in raw:
        h = h.strip()
        if h and h not in seen:
            seen.add(h); out.append(h)
    return out

def fetch_newsdata(query, api_key, max_results=10):
    try:
        r = requests.get("https://newsdata.io/api/1/news",
                        params={"apikey": api_key, "q": query,
                                "language": "en", "category": "business",
                                "size": max_results}, timeout=15)
        r.raise_for_status()
        data = r.json()
        if data.get("status") == "success":
            return [a.get("title") or a.get("description", "") for a in data.get("results", []) if a.get("title")]
    except Exception as e:
        st.warning(f"NewsData.io error: {e}")
    return []

def fetch_gnews(query, api_key, max_results=10):
    try:
        r = requests.get("https://gnews.io/api/v4/search",
                        params={"token": api_key, "q": query, "lang": "en",
                                "country": "in", "max": max_results,
                                "sortBy": "publishedAt"}, timeout=15)
        r.raise_for_status()
        return [a.get("title") or a.get("description", "") for a in r.json().get("articles", []) if a.get("title")]
    except Exception as e:
        st.warning(f"GNews error: {e}")
    return []

def compute_rsi(close, period=14):
    delta = close.diff()
    gain = delta.clip(lower=0)
    loss = (-delta).clip(lower=0)
    avg_g = gain.ewm(alpha=1/period, min_periods=period).mean()
    avg_l = loss.ewm(alpha=1/period, min_periods=period).mean()
    rs = avg_g / avg_l.replace(0, np.nan)
    return 100 - 100 / (1 + rs)

def engineer_features(ohlc, macro_df, start_date, end_date):
    import numpy as np
    df = ohlc.copy()
    df["Returns"] = df["Close"].pct_change()

```

```

df["Log_Returns"] = np.log(df["Close"] / df["Close"].shift(1))
df["RSI"] = compute_rsi(df["Close"])
ema12 = df["Close"].ewm(span=12, adjust=False).mean()
ema26 = df["Close"].ewm(span=26, adjust=False).mean()
df["MACD"] = ema12 - ema26
df["MACD_Signal"] = df["MACD"].ewm(span=9, adjust=False).mean()
df["MACD_Hist"] = df["MACD"] - df["MACD_Signal"]
bb_mid = df["Close"].rolling(20).mean()
bb_std = df["Close"].rolling(20).std()
df["BB_Upper"] = bb_mid + 2*bb_std
df["BB_Lower"] = bb_mid - 2*bb_std
df["BB_Width"] = (df["BB_Upper"] - df["BB_Lower"]) / bb_mid
df["BB_PctB"] = (df["Close"] - df["BB_Lower"]) / (df["BB_Upper"] - df["BB_Lower"] + 1e-9)
hl = df["High"] - df["Low"]
hc = (df["High"] - df["Close"].shift()).abs()
lc = (df["Low"] - df["Close"].shift()).abs()
df["ATR"] = pd.concat([hl, hc, lc], axis=1).max(axis=1).ewm(span=14, adjust=False).mean()
df["ATR_Pct"] = df["ATR"] / df["Close"]
df["OBV"] = (np.sign(df["Close"].diff().fillna(0)) * df["Volume"]).cumsum()
for w in [7, 14, 21, 50]:
    df[f"Roll_Mean_{w}"] = df["Close"].rolling(w).mean()
    df[f"Roll_Std_{w}"] = df["Close"].rolling(w).std()
    df[f"RVol_{w}"] = df["Returns"].rolling(w).std() * np.sqrt(252)
df["Momentum_5"] = df["Close"] / df["Close"].shift(5) - 1
df["Momentum_20"] = df["Close"] / df["Close"].shift(20) - 1
df["HL_Spread"] = (df["High"] - df["Low"]) / df["Close"]
df["CO_Spread"] = (df["Close"] - df["Open"]) / df["Open"]
low14 = df["Low"].rolling(14).min()
high14 = df["High"].rolling(14).max()
df["Stoch_K"] = 100*(df["Close"] - low14) / (high14 - low14 + 1e-9)
df["Stoch_D"] = df["Stoch_K"].rolling(3).mean()
df["Williams_R"] = -100*(high14 - df["Close"]) / (high14 - low14 + 1e-9)
df["Vol_MA_20"] = df["Volume"].rolling(20).mean()
df["Vol_Ratio"] = df["Volume"] / (df["Vol_MA_20"] + 1)
for lag in [1, 2, 3, 5, 10]:
    df[f"Ret_Lag_{lag}"] = df["Returns"].shift(lag)
    df[f"Close_Lag_{lag}"] = df["Close"].shift(lag)
df["DayOfWeek"] = df.index.dayofweek
df["Month"] = df.index.month
df["Quarter"] = df.index.quarter
ema20 = df["Close"].ewm(span=20, adjust=False).mean()
ema50 = df["Close"].ewm(span=50, adjust=False).mean()
df["Regime"] = np.where(ema20 > ema50, 1, -1)
if not macro_df.empty:
    for col in macro_df.columns:
        df[f"{col}_Ret"] = macro_df[col].pct_change()
for col in ["Returns", "Log_Returns"]:
    q1, q3 = df[col].quantile(0.01), df[col].quantile(0.99)
    df[col] = df[col].clip(q1, q3)
df["Target"] = (df["Close"].shift(-1) > df["Close"]).astype(int)
df["Next_Close"] = df["Close"].shift(-1)
df.ffill(inplace=True); df.bfill(inplace=True); df.dropna(inplace=True)
return df

def run_prophet(df, ohlcv):
    regressors = ["RSI", "MACD", "MACD_Hist", "BB_Width",
                  "ATR_Pct", "Momentum_5", "Momentum_20",
                  "Vol_Ratio", "HL_Spread", "CO_Spread"]
    df_p = pd.DataFrame({
        "ds": pd.to_datetime(ohlcv.index[:len(df)]),
        "y": (df["Log_Returns"] - df["Log_Returns"].rolling(5).mean()).values
    }).reset_index(drop=True)
    for r in regressors:
        if r in df.columns:
            df_p[r] = df[r].values
    split = int(len(df_p) * 0.70)
    train = df_p.iloc[:split]
    m = Prophet(changepoint_prior_scale=0.1, seasonality_mode="multiplicative",
                yearly_seasonality=True, weekly_seasonality=True, daily_seasonality=False)
    for r in regressors:
        if r in train.columns:
            m.add_regressor(r)
    m.fit(train)
    forecast = m.predict(df_p)
    test_f = forecast.iloc[split:]
    test_a = df_p.iloc[split:]

```

```

p_dir = (test_f["yhat"].values > 0).astype(int)
a_dir = (test_a["y"].values > 0).astype(int)
p_acc = float(np.mean(p_dir == a_dir))
df["Prophet_Return"] = forecast["yhat"].values[:len(df)]
df["Prophet_Trend"] = forecast["trend"].values[:len(df)]
df["Prophet_Upper"] = forecast["yhat_upper"].values[:len(df)]
df["Prophet_Lower"] = forecast["yhat_lower"].values[:len(df)]
df["Prophet_Confidence"] = (1 / (forecast["yhat_upper"].values[:len(df)]
                                - forecast["yhat_lower"].values[:len(df)] + 1e-6))

df["Prophet_Direction"] = (df["Prophet_Return"].shift(-1) > 0).astype(int)
current_price = float(df["Close"].iloc[-1])
next_ret = float(df["Prophet_Return"].iloc[-1])
prophet_price = float(current_price * np.exp(next_ret))
return df, p_acc, prophet_price

def run_arima_sarima(df):
    close_series = df["Close"].copy()
    log_close = np.log(close_series)
    train_end = int(len(log_close) * 0.80)
    train_log = log_close.iloc[:train_end]
    test_log = log_close.iloc[train_end:]
    arima = auto_arima(train_log, seasonal=False, stepwise=True,
                      suppress_warnings=True, error_action="ignore",
                      max_p=5, max_q=5, max_d=2, information_criterion="aic")
    ap_log = arima.predict(n_periods=len(test_log))
    ap = np.exp(ap_log)
    aa = np.exp(test_log.values)
    a_dir = float(np.mean(np.sign(np.diff(ap)) == np.sign(np.diff(aa)))))
    an_log = float(arima.predict(n_periods=1).iloc[0])
    a_price = float(np.exp(an_log))
    try:
        sarima = auto_arima(train_log, seasonal=True, m=5, stepwise=True,
                          suppress_warnings=True, error_action="ignore",
                          max_p=3, max_q=3, max_d=2,
                          max_P=2, max_Q=2, max_D=1, information_criterion="aic")
        sp_log = sarima.predict(n_periods=len(test_log))
        sp = np.exp(sp_log)
        s_dir = float(np.mean(np.sign(np.diff(sp)) == np.sign(np.diff(aa)))))
        sn_log = sarima.predict(n_periods=1)[0]
        s_price = float(np.exp(sn_log))
    except Exception:
        s_dir = a_dir
        s_price = a_price
    return a_dir, a_price, s_dir, s_price

def boost_features(df):
    feat = df.copy()
    for w in [5, 10, 20, 50, 200]:
        ma = feat["Close"].rolling(w).mean()
        feat[f"Price_vs_MA{w}"] = (feat["Close"] - ma) / ma
    feat["RSI_OB"] = (feat["RSI"] > 65).astype(int)
    feat["RSI_OS"] = (feat["RSI"] < 35).astype(int)
    feat["RSI_Momentum"] = feat["RSI"] - feat["RSI"].shift(3)
    feat["MACD_Cross"] = np.sign(feat["MACD"] - feat["MACD_Signal"])
    feat["MACD_Cross_Chg"] = feat["MACD_Cross"].diff()
    feat["BB_Squeeze"] = (feat["BB_Width"] < feat["BB_Width"].rolling(20).mean()).astype(int)
    feat["BB_Position"] = feat["BB_PctB"].clip(0, 1)
    feat["Vol_Surge"] = (feat["Volume"] > feat["Volume"].rolling(20).mean()*1.5).astype(int)
    feat["Vol_Trend"] = feat["Volume"].rolling(5).mean() / feat["Volume"].rolling(20).mean()
    feat["Up_Days"] = feat["Returns"].gt(0).astype(int)
    feat["Ret_Accel"] = feat["Returns"] - feat["Returns"].shift(1)
    feat["Ret_3d_Sum"] = feat["Returns"].rolling(3).sum()
    feat["Ret_5d_Sum"] = feat["Returns"].rolling(5).sum()
    feat["Ret_10d_Sum"] = feat["Returns"].rolling(10).sum()
    feat["Gap"] = (feat["Open"] - feat["Close"].shift(1)) / feat["Close"].shift(1)
    feat["ATR_Move"] = feat["Returns"].abs() / (feat["ATR_Pct"] + 1e-9)
    feat["Stoch_Cross"] = np.sign(feat["Stoch_K"] - feat["Stoch_D"])
    high52 = feat["High"].rolling(252).max()
    low52 = feat["Low"].rolling(252).min()
    feat["Dist_52w_High"] = (feat["Close"] - high52) / high52
    feat["Dist_52w_Low"] = (feat["Close"] - low52) / low52
    feat["ADX_Proxy"] = feat["ATR"].rolling(14).mean() / feat["Close"]
    feat.ffill(inplace=True); feat.bfill(inplace=True); feat.dropna(inplace=True)
    COLS = [
        "Price_vs_MA5", "Price_vs_MA10", "Price_vs_MA20", "Price_vs_MA50", "Price_vs_MA200",

```

```

    "RSI", "RSI_OB", "RSI_OS", "RSI_Momentum",
    "MACD", "MACD_Signal", "MACD_Hist", "MACD_Cross", "MACD_Cross_Chg",
    "BB_PctB", "BB_Width", "BB_Squeeze", "BB_Position",
    "Vol_Ratio", "Vol_Surge", "Vol_Trend",
    "Returns", "Ret_3d_Sum", "Ret_5d_Sum", "Ret_10d_Sum", "Ret_Accel",
    "Ret_Lag_1", "Ret_Lag_2", "Ret_Lag_3", "Ret_Lag_5",
    "Momentum_5", "Momentum_20", "Gap",
    "ATR_Pct", "ATR_Move", "HL_Spread", "CO_Spread",
    "Stoch_K", "Stoch_D", "Stoch_Cross", "Williams_R",
    "Dist_52w_High", "Dist_52w_Low", "Regime",
    "DayOfWeek", "Month", "Quarter",
]
COLS = [c for c in COLS if c in feat.columns]
return feat, COLS

def run_ensemble(feat, COLS):
    from sklearn.metrics import accuracy_score, precision_score, recall_score
    from sklearn.model_selection import TimeSeriesSplit
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.preprocessing import RobustScaler
    import xgboost as xgb
    import lightgbm as lgb

    fc = feat[COLS + ["Target"]].dropna().copy()
    X = fc[COLS].values.astype(np.float32)
    y = fc["Target"].values.astype(int)
    n = len(X)
    n_tr = int(n*0.70); n_vl = int(n*0.10)
    X_tr, X_vl, X_te = X[:n_tr], X[n_tr:n_tr+n_vl], X[n_tr+n_vl:]
    y_tr, y_vl, y_te = y[:n_tr], y[n_tr:n_tr+n_vl], y[n_tr+n_vl:]
    sc = RobustScaler()
    Xtr_s = sc.fit_transform(X_tr)
    Xvl_s = sc.transform(X_vl)
    Xte_s = sc.transform(X_te)
    xgb1 = xgb.XGBClassifier(n_estimators=400, max_depth=5, learning_rate=0.03,
                             subsample=0.75, colsample_bytree=0.75,
                             eval_metric="logloss", use_label_encoder=False,
                             random_state=42, verbosity=0)
    xgb1.fit(Xtr_s, y_tr, eval_set=[(Xvl_s, y_vl)], verbose=False)
    xgb_acc = accuracy_score(y_te, xgb1.predict(Xte_s))
    lgb1 = lgb.LGBMClassifier(n_estimators=400, max_depth=5, learning_rate=0.03,
                              subsample=0.75, colsample_bytree=0.75,
                              random_state=42, verbose=-1)
    lgb1.fit(Xtr_s, y_tr, eval_set=[(Xvl_s, y_vl)],
             callbacks=[lgb.early_stopping(50, verbose=False), lgb.log_evaluation(-1)])
    lgb_acc = accuracy_score(y_te, lgb1.predict(Xte_s))
    rf1 = RandomForestClassifier(n_estimators=200, max_depth=6, min_samples_leaf=10,
                                max_features="sqrt", class_weight="balanced",
                                random_state=42, n_jobs=-1)
    rf1.fit(Xtr_s, y_tr)
    rf_acc = accuracy_score(y_te, rf1.predict(Xte_s))
    w_total = xgb_acc + lgb_acc + rf_acc
    pb_xgb = xgb1.predict_proba(Xte_s)[: , 1]
    pb_lgb = lgb1.predict_proba(Xte_s)[: , 1]
    pb_rf = rf1.predict_proba(Xte_s)[: , 1]
    prob_ens = (pb_xgb*xgb_acc + pb_lgb*lgb_acc + pb_rf*rf_acc) / w_total
    pred_ens = (prob_ens >= 0.50).astype(int)
    ens_acc = accuracy_score(y_te, pred_ens)
    ens_prec = precision_score(y_te, pred_ens, zero_division=0)
    ens_rec = recall_score(y_te, pred_ens, zero_division=0)
    tscv = TimeSeriesSplit(n_splits=5)
    wf_accs = []
    for tr_idx, te_idx in tscv.split(X):
        Xtr_f = sc.fit_transform(X[tr_idx])
        Xte_f = sc.transform(X[te_idx])
        clf = xgb.XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.03,
                                eval_metric="logloss", use_label_encoder=False,
                                random_state=42, verbosity=0)
        clf.fit(Xtr_f, y[tr_idx])
        wf_accs.append(accuracy_score(y[te_idx], clf.predict(Xte_f)))
    wf_mean = float(np.mean(wf_accs))
    live_f = feat[COLS].iloc[-1:].values.astype(np.float32)
    live_s = sc.transform(live_f)
    live_prob = float((xgb1.predict_proba(live_s)[0,1]*xgb_acc +
                       lgb1.predict_proba(live_s)[0,1]*lgb_acc +
                       rf1.predict_proba(live_s)[0,1]*rf_acc) / w_total)

```

```

return {"xgb_acc": xgb_acc, "lgb_acc": lgb_acc, "rf_acc": rf_acc,
        "ens_acc": ens_acc, "ens_prec": ens_prec, "ens_rec": ens_rec,
        "wf_mean": wf_mean, "live_prob_ens": live_prob,
        "sc": sc, "feat_cols": COLS}

def compute_sentiment(headlines):
    from textblob import TextBlob
    tb_scores = [TextBlob(h).sentiment.polarity for h in headlines]
    tb_score = float(np.mean(tb_scores)) if headlines else 0.0
    fb_score = 0.0
    try:
        from transformers import pipeline as hf_pipeline
        LABEL_MAP = {"positive": 1, "neutral": 0, "negative": -1}
        finbert = hf_pipeline("text-classification", model="ProsusAI/finbert",
                              top_k=None, truncation=True, max_length=512)

        scores = []
        for h in headlines[:10]:
            res = finbert(h)[0]
            w = sum(LABEL_MAP.get(r["label"].lower(), 0)*r["score"] for r in res)
            scores.append(w)
        fb_score = float(np.mean(scores))
    except Exception:
        fb_score = tb_score
    return tb_score, fb_score

def ask_rag(question, embedder, index, documents, groq_client, company_name):
    q_vec = embedder.encode([question], normalize_embeddings=True).astype(np.float32)
    _, ids = index.search(q_vec, k=5) # bumped k from 3 to 5 since we have more docs now
    context = "\n\n".join([documents[i] for i in ids[0] if i < len(documents)])
    resp = groq_client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[
            {"role": "system", "content":
             f"You are a stock analyst for {company_name}. "
             "Answer only using the context provided. Be concise and specific."},
            {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"}
        ],
        temperature=0.2, max_tokens=400
    )
    return resp.choices[0].message.content.strip()

def price_signal(pred, cur, thresh=1.0):
    pct = (pred - cur) / cur * 100
    return "BUY" if pct >= thresh else "SELL" if pct <= -thresh else "HOLD"

def chunk_text(text, size=500, overlap=100):
    chunks, start = [], 0
    while start < len(text):
        chunks.append(text[start:start+size].strip())
        start += size - overlap
    return [c for c in chunks if len(c) > 50]

if run_btn:
    if not company_name or not ticker:
        st.sidebar.error("Please enter Company Name and Ticker.")
    else:
        st.session_state.company_name = company_name
        st.session_state.ticker = ticker
        st.session_state.start_date = str(start_date)
        st.session_state.end_date = str(end_date)

        with st.spinner("Importing libraries..."):
            lazy_import()

        progress = st.progress(0, text="Starting...")

        # News
        progress.progress(5, "Fetching news headlines...")
        q = build_query(company_name, ticker)
        nd, gn = [], []
        if news_source != "GNews only":
            nd = fetch_newsdata(q, newsdata_key)
        if news_source != "NewsData.io only":

```

```

    gn = fetch_gnews(q, gnews_key)
    st.session_state.headlines = clean_headlines(nd + gn)

# Screener.in Auto-Fetch
progress.progress(12, "Fetching Screener.in data...")
screener_data = fetch_screener_data(ticker)
st.session_state.screener_data = screener_data
st.session_state.screener_fetched = True
st.session_state.documents_extra = (
    [screener_data["about"]] if screener_data.get("about") else []
) + screener_data.get("pros", []) + screener_data.get("cons", [])

# Stock Data
progress.progress(15, "Downloading OHLCV data...")
import yfinance as yf
raw = yf.download(ticker, start=str(start_date), end=str(end_date),
                  auto_adjust=True, progress=False)
if isinstance(raw.columns, pd.MultiIndex):
    raw.columns = raw.columns.get_level_values(0)
ohlc = raw[["Open", "High", "Low", "Close", "Volume"]].copy()
ohlc.index = pd.to_datetime(ohlc.index).tz_localize(None)
ohlc.dropna(inplace=True)
st.session_state.ohlc = ohlc

if ohlc.empty:
    st.error("No data returned. Check ticker symbol.")
    st.stop()

# Macro
macro_syms = {"^NSEI": "Nifty50", "USDINR=X": "USDINR", "GC=F": "Gold"}
macro_df = pd.DataFrame()
for sym, name in macro_syms.items():
    try:
        m = yf.download(sym, start=str(start_date), end=str(end_date),
                        auto_adjust=True, progress=False)
        if isinstance(m.columns, pd.MultiIndex):
            m.columns = m.columns.get_level_values(0)
        s = m["Close"].rename(name)
        s.index = pd.to_datetime(s.index).tz_localize(None)
        macro_df[name] = s
    except Exception:
        pass
macro_df = macro_df.reindex(ohlc.index, method="ffill").ffill().bfill()

# Feature Engineering
progress.progress(25, "Engineering features...")
df = engineer_features(ohlc, macro_df, str(start_date), str(end_date))
current_price = float(df["Close"].iloc[-1])

# Prophet
progress.progress(35, "Running Prophet...")
from prophet import Prophet as _Prophet
df, prophet_acc, prophet_price = run_prophet(df, ohlc)

# ARIMA / SARIMA
progress.progress(50, "Fitting ARIMA/SARIMA...")
from pmdarima import auto_arima as _auto_arima
arima_acc, arima_price, sarima_acc, sarima_price = run_arima_sarima(df)

# Boosted Ensemble
progress.progress(65, "Training XGB/LGB/RF ensemble...")
feat_df, BOOST_COLS = boost_features(df)
ens_res = run_ensemble(feat_df, BOOST_COLS)
live_prob_ens = ens_res["live_prob_ens"]
wf_mean = ens_res["wf_mean"]
ens_acc = ens_res["ens_acc"]

# Sentiment
progress.progress(78, "Running sentiment analysis...")
headlines = st.session_state.headlines
tb_score, fb_score = compute_sentiment(headlines)

bull_words = ["growth", "positive", "strong", "bullish", "increase", "profit", "revenue"]
bear_words = ["risk", "decline", "bearish", "negative", "loss", "weak", "concern"]
dummy_ans = " ".join(headlines).lower()
bc = sum(1 for w in bull_words if w in dummy_ans)
br = sum(1 for w in bear_words if w in dummy_ans)

```

```

rag_score = float(np.clip((bc - br) / (bc + br + 1e-9), -1, 1))

# RAG Index now includes Screener data
progress.progress(88, "Building RAG knowledge base...")

# Start with PDF summary (auto-fetched or previously uploaded)
documents = chunk_text(st.session_state.pdf_summary) if st.session_state.pdf_summary else []

# Add Screener.in raw text (ratios, pros, cons, financials)
screener_data = st.session_state.get("screener_data", {})
if screener_data.get("raw_text"):
    documents.extend(chunk_text(screener_data["raw_text"], size=600, overlap=100))

# Add news headlines
documents.extend(headlines)
documents.extend(st.session_state.get("documents_extra", []))

if not documents:
    documents = [f"{company_name} stock analysis context."]

from sentence_transformers import SentenceTransformer
import faiss as _faiss
embedder = SentenceTransformer("all-MiniLM-L6-v2")
vecs = embedder.encode(documents, normalize_embeddings=True).astype(np.float32)
idx = _faiss.IndexFlatIP(vecs.shape[1])
idx.add(vecs)
st.session_state.faiss_index = idx
st.session_state.embedder = embedder
st.session_state.documents = documents

from groq import Groq
st.session_state.groq_client = Groq(api_key=groq_key)

# Final Signal
progress.progress(95, "Computing final signal...")
a_sig = price_signal(arima_price, current_price)
sa_sig = price_signal(sarima_price, current_price)
pr_sig = price_signal(prophet_price, current_price)
ens_sig = "BUY" if live_prob_ens >= 0.60 else "SELL" if live_prob_ens <= 0.40 else "HOLD"

all_sigs = [a_sig, sa_sig, pr_sig, ens_sig]
buy_c = all_sigs.count("BUY")
sell_c = all_sigs.count("SELL")
majority = "BUY" if buy_c > sell_c else "SELL" if sell_c > buy_c else "HOLD"

if fb_score > 0.3 and majority == "HOLD": majority = "BUY"
elif fb_score < -0.3 and majority == "HOLD": majority = "SELL"
if rag_score > 0.6: majority = "BUY"
elif rag_score < -0.6: majority = "SELL"
regime_val = int(df["Regime"].iloc[-1])
if regime_val == -1 and majority == "BUY": majority = "HOLD"

agreement = max(buy_c, sell_c) / len(all_sigs)
latest = df.iloc[-1]
tech_sigs = []
if latest["RSI"] < 30: tech_sigs.append("BUY")
elif latest["RSI"] > 70: tech_sigs.append("SELL")
else: tech_sigs.append("HOLD")
if latest["MACD"] > latest["MACD_Signal"]: tech_sigs.append("BUY")
else: tech_sigs.append("SELL")
if latest["Stoch_K"] > latest["Stoch_D"]: tech_sigs.append("BUY")
else: tech_sigs.append("SELL")
tech_agreement = tech_sigs.count(majority) / len(tech_sigs)

meta_conf = abs(live_prob_ens - 0.5) * 2
sent_avg = (fb_score + tb_score) / 2
sent_conf = max(0, sent_avg) if majority == "BUY" else max(0, -sent_avg) if majority == "SELL" else 1.0 - abs(sent_avg)
confidence = min(0.25*meta_conf + 0.20*agreement + 0.15*tech_agreement +
                 0.10*sent_conf + 0.10*min(wf_mean, 1.0), 0.95)

prices_flat = df["Close"].values
running_max = np.maximum.accumulate(prices_flat)
max_dd = float(((prices_flat - running_max) / running_max * 100).min())
ann_vol = float(df["Returns"].std() * np.sqrt(252) * 100)
sharpe = float((df["Returns"].mean()*252 - 0.065) / (df["Returns"].std()*np.sqrt(252) + 1e-9))
risk_level = "LOW" if abs(max_dd) < 10 else "MEDIUM" if abs(max_dd) < 25 else "HIGH"

```

```

ensemble_price = float(np.mean([arima_price, sarima_price, prophet_price]))

st.session_state.df_feat = df
st.session_state.models_trained = True
st.session_state.results = {
    "current_price": current_price,
    "ensemble_price": ensemble_price,
    "arima_price": arima_price,
    "sarima_price": sarima_price,
    "prophet_price": prophet_price,
    "arima_acc": arima_acc,
    "sarima_acc": sarima_acc,
    "prophet_acc": prophet_acc,
    "ens_acc": ens_acc,
    "wf_mean": wf_mean,
    "live_prob_ens": live_prob_ens,
    "tb_score": tb_score,
    "fb_score": fb_score,
    "rag_score": rag_score,
    "majority": majority,
    "confidence": confidence,
    "max_dd": max_dd,
    "ann_vol": ann_vol,
    "sharpe": sharpe,
    "risk_level": risk_level,
    "regime": regime_val,
    "a_sig": a_sig, "sa_sig": sa_sig, "pr_sig": pr_sig, "ens_sig": ens_sig,
}
progress.progress(100, "Done!")
time.sleep(0.4)
progress.empty()
st.success("✅ Analysis complete!")

tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs([
    "🏠 Dashboard", "📊 Charts", "🧠 Models", "📰 Sentiment", "🔍 Screener Data", "💬 RAG Chatbot"
])

# DASHBOARD
with tab1:
    if not st.session_state.models_trained:
        st.info("👉 Enter company details in the sidebar and click **Run Full Analysis** to begin.")
        st.markdown("""
        ### What this app does
        - **News Fetching**: pulls latest headlines from NewsData.io and GNews
        - **Screener.in Auto-Fetch**: automatically pulls ratios, pros/cons, financials & annual report PDF
        - **Technical Indicators**: RSI, MACD, Bollinger Bands, ATR, OBV, Stochastics
        - **Models**: ARIMA, SARIMA, Prophet, XGBoost, LightGBM, Random Forest
        - **Sentiment**: TextBlob + FinBERT on news headlines
        - **Final Signal**: ensemble BUY / SELL / HOLD with confidence score
        - **RAG Chatbot**: ask any question grounded in Screener data + annual report + news
        """)
    else:
        r = st.session_state.results
        cn = st.session_state.company_name
        tk = st.session_state.ticker

        st.title(f"🏠 {cn} ({tk})")
        st.caption(f"Analysis as of {datetime.now().strftime('%Y-%m-%d %H:%M')}")

        sig_class = {"BUY": "signal-buy", "SELL": "signal-sell", "HOLD": "signal-hold"}[r["majority"]]
        st.markdown(f"""
        <div class="metric-card">
            <span style="color:#aaa">FINAL SIGNAL</span><br>
            <span class="{sig_class}">{r['majority']}</span>
            &nbsp;&nbsp;&nbsp;
            <span style="color:#ccc;font-size:1rem">Confidence: <b>{r['confidence']:.1%}</b></span>
            &nbsp;&nbsp;&nbsp;
            <span style="color:#ccc;font-size:.9rem">P(UP): {r['live_prob_ens']:.2%}</span>
            &nbsp;&nbsp;&nbsp;
            <span style="color:#ccc;font-size:.9rem">Regime: {'🐮 BULL' if r['regime']==1 else '🐻 BEAR'}</span>
        </div>
        """, unsafe_allow_html=True)

        c1, c2, c3, c4 = st.columns(4)
        c1.metric("Current Price", f"₹{r['current_price']:.2f}")

```

```

c2.metric("Ensemble Target", f"{r['ensemble_price']:.2f}",
          f"({r['ensemble_price']-r['current_price']}/r['current_price']*100:+.2f}%")
c3.metric("Sharpe Ratio", f"{r['sharpe']:.3f}")
c4.metric("Risk Level", r["risk_level"])

st.markdown("#### Individual Model Signals")
sc1, sc2, sc3, sc4 = st.columns(4)
for col, label, sig, price in [
    (sc1, "ARIMA", r["a_sig"], r["arima_price"]),
    (sc2, "SARIMA", r["sa_sig"], r["sarima_price"]),
    (sc3, "Prophet", r["pr_sig"], r["prophet_price"]),
    (sc4, "Ensemble", r["ens_sig"], r["ensemble_price"]),
]:
    color = "#26a69a" if sig=="BUY" else "#ef5350" if sig=="SELL" else "#ffd700"
    col.markdown(f"""
<div class="metric-card">
    <span style="color:#aaa;font-size:.8rem">{label}</span><br>
    <span style="color:{color};font-size:1.2rem;font-weight:700">{sig}</span><br>
    <span style="color:#ccc;font-size:.85rem">{price:.2f}</span>
</div>""", unsafe_allow_html=True)

st.markdown("#### Risk Metrics")
r1, r2, r3, r4 = st.columns(4)
r1.metric("Max Drawdown", f"{r['max_dd']:.2f}%")
r2.metric("Ann. Volatility", f"{r['ann_vol']:.2f}%")
r3.metric("Walk-Fwd Acc", f"{r['wf_mean']:.2f}%")
r4.metric("Ensemble Acc", f"{r['ens_acc']:.2f}%")

# CHARTS
with tab2:
    if st.session_state.df_feat is not None:
        df = st.session_state.df_feat
        cn = st.session_state.company_name
        tk = st.session_state.ticker
        window = st.slider("Days to display", 60, min(len(df), 756), 252, 30)
        plot = df.tail(window).copy()

        fig = make_subplots(rows=4, cols=1, shared_xaxes=True,
                             row_heights=[0.45, 0.2, 0.2, 0.15],
                             subplot_titles=["Price + Bollinger Bands", "RSI (14)", "MACD", "Volume"],
                             vertical_spacing=0.06)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["Close"], name="Close",
                                  line=dict(color="#00d4ff", width=2)), row=1, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["BB_Upper"], name="BB Upper",
                                  line=dict(color="steelblue", dash="dash", width=0.9)), row=1, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["BB_Lower"], name="BB Lower",
                                  line=dict(color="steelblue", dash="dash", width=0.9),
                                  fill="tonexty", fillcolor="rgba(70,130,180,0.07)"), row=1, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["Roll_Mean_21"], name="21d MA",
                                  line=dict(color="#ffd700", width=1)), row=1, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["RSI"], name="RSI",
                                  line=dict(color="#ff6b6b", width=1.5)), row=2, col=1)
        fig.add_hline(y=70, line_color="red", line_dash="dash", row=2, col=1)
        fig.add_hline(y=30, line_color="green", line_dash="dash", row=2, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["MACD"], name="MACD",
                                  line=dict(color="#00d4ff")), row=3, col=1)
        fig.add_trace(go.Scatter(x=plot.index, y=plot["MACD_Signal"], name="Signal",
                                  line=dict(color="#ff6b6b")), row=3, col=1)
        colors_hist = ["#26a69a" if v >= 0 else "#ef5350" for v in plot["MACD_Hist"]]
        fig.add_trace(go.Bar(x=plot.index, y=plot["MACD_Hist"], name="Hist",
                              marker_color=colors_hist, opacity=0.7), row=3, col=1)
        vol_colors = ["#26a69a" if r >= 0 else "#ef5350" for r in plot["Returns"]]
        fig.add_trace(go.Bar(x=plot.index, y=plot["Volume"], name="Volume",
                              marker_color=vol_colors, opacity=0.7), row=4, col=1)
        fig.update_layout(template="plotly_dark", height=760,
                             title=f"{cn} ({tk}) - Technical Dashboard",
                             showlegend=True, hovermode="x unified",
                             legend=dict(orientation="h", yanchor="bottom", y=1.01))
        fig.update_yaxes(range=[0, 100], row=2, col=1)
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.info("Run analysis first to see charts.")

# MODELS
with tab3:

```

```

if st.session_state.models_trained:
    r = st.session_state.results
    cn = st.session_state.company_name
    st.subheader("Model Performance Summary")
    models = ["ARIMA", "SARIMA", "Prophet", "XGB/LGB/RF Ensemble"]
    accs = [r["arima_acc"]*100, r["sarima_acc"]*100, r["prophet_acc"]*100, r["ens_acc"]*100]
    fig_cmp = go.Figure(go.Bar(
        x=models, y=accs,
        marker_color=["#26a69a" if a >= 55 else "#ef5350" for a in accs],
        text=[f"{a:.1f}%" for a in accs], textposition="outside"
    ))
    fig_cmp.add_hline(y=55, line_dash="dash", line_color="#ffd700",
        annotation_text="Target 55%", annotation_font_color="#ffd700")
    fig_cmp.update_layout(template="plotly_dark", height=380,
        title="Directional Accuracy Comparison",
        yaxis=dict(title="Accuracy %", range=[0, 100]))
    st.plotly_chart(fig_cmp, use_container_width=True)
    st.markdown("#### Price Forecasts")
    col_a, col_b, col_c = st.columns(3)
    col_a.metric("ARIMA", f"{r['arima_price']:.2f}",
        f"({r['arima_price']-r['current_price']}/r['current_price']*100:+.2f}%")
    col_b.metric("SARIMA", f"{r['sarima_price']:.2f}",
        f"({r['sarima_price']-r['current_price']}/r['current_price']*100:+.2f}%")
    col_c.metric("Prophet", f"{r['prophet_price']:.2f}",
        f"({r['prophet_price']-r['current_price']}/r['current_price']*100:+.2f}%")
    st.markdown("#### Walk-Forward Validation (5-fold)")
    st.metric("Mean Walk-Forward Accuracy", f"{r['wf_mean']:.2%}")
    st.caption("Uses time-series cross-validation to prevent data leakage.")
    st.markdown("#### Ensemble Classifier")
    e1, e2, e3 = st.columns(3)
    e1.metric("Ensemble Acc", f"{r['ens_acc']:.2%}")
    e2.metric("P(UP)", f"{r['live_prob_ens']:.2%}")
    e3.metric("Signal", r["ens_sig"])
else:
    st.info("Run analysis first.")

# SENTIMENT
with tab4:
    if st.session_state.models_trained:
        r = st.session_state.results
        st.subheader("Sentiment Analysis")
        c1, c2, c3 = st.columns(3)
        c1.metric("TextBlob Score", f"{r['tb_score']:.4f}",
            "Positive" if r["tb_score"] > 0 else "Negative")
        c2.metric("FinBERT Score", f"{r['fb_score']:.4f}",
            "Positive" if r["fb_score"] > 0 else "Negative")
        c3.metric("RAG Score", f"{r['rag_score']:.4f}",
            "Bullish" if r["rag_score"] > 0 else "Bearish")
        headlines = st.session_state.headlines
        st.markdown(f"#### News Headlines ({len(headlines)} fetched)")
        if headlines:
            for i, h in enumerate(headlines, 1):
                st.markdown(f"**{i}.** {h}")
        else:
            st.warning("No headlines fetched. Check API keys and company name.")
    else:
        st.info("Run analysis first.")

# SCREENER DATA (replaces old Annual Report tab)
with tab5:
    st.subheader("🔍 Screener.in – Auto-Fetched Company Data")

    if not st.session_state.screener_fetched:
        st.info("Run analysis to auto-fetch Screener.in data for this company.")

    st.markdown("#### Or upload Annual Report PDF manually")
    uploaded = st.file_uploader("Upload Annual Report PDF", type=["pdf"])
    if uploaded:
        with st.spinner("Extracting & summarising PDF..."):
            try:
                import pdfplumber, tempfile
                from groq import Groq
                tmp = tempfile.NamedTemporaryFile(suffix=".pdf", delete=False)
                tmp.write(uploaded.read()); tmp.flush()
                pages = []

```

```

        with pdfplumber.open(tmp.name) as pdf:
            for i, page in enumerate(pdf.pages):
                if i >= 20: break
                txt = page.extract_text()
                if txt: pages.append(txt)
            full_text = "\n\n".join(pages)
            client = Groq(api_key=groq_key)
            chunks = [full_text[i:i+15000] for i in range(0, min(len(full_text), 60000), 15000)]
            chunk_summaries = []
            for chunk in chunks:
                resp = client.chat.completions.create(
                    model="llama-3.1-8b-instant",
                    messages=[{"role": "user", "content":
                        f"Summarise this annual report section in bullet points "
                        f"(revenue, profit, risks, outlook):\n\n{chunk}"}],
                    temperature=0.2, max_tokens=512
                )
                chunk_summaries.append(resp.choices[0].message.content.strip())
            final_resp = client.chat.completions.create(
                model="llama-3.1-8b-instant",
                messages=[{"role": "user", "content":
                    "Combine these into one executive summary with "
                    "Financial Performance, Business Highlights, Risks, Outlook:\n\n" +
                    "\n---\n".join(chunk_summaries)}],
                temperature=0.2, max_tokens=800
            )
            summary = final_resp.choices[0].message.content.strip()
            st.session_state.pdf_summary = summary
            st.success("PDF summarised and added to knowledge base! Re-run analysis to include it.")
            st.markdown(summary)
        except Exception as e:
            st.error(f"PDF processing failed: {e}")
    else:
        sd = st.session_state.screener_data
        cn = st.session_state.company_name

        st.markdown(f"***Source:** [{sd['url']}]({sd['url']})")

        # About
        if sd.get("about"):
            st.markdown(f"#### About {cn}")
            st.write(sd["about"])

        # Key Ratios
        if sd.get("ratios"):
            st.markdown("#### 📊 Key Ratios")
            ratio_items = list(sd["ratios"].items())
            cols = st.columns(4)
            for i, (k, v) in enumerate(ratio_items):
                cols[i % 4].metric(k, v)

        # Pros & Cons
        col_p, col_c = st.columns(2)
        with col_p:
            if sd.get("pros"):
                st.markdown("#### Strengths")
                for p in sd["pros"]:
                    st.markdown(f"- {p}")
            else:
                st.markdown("#### Strengths")
                st.caption("None found on page.")
        with col_c:
            if sd.get("cons"):
                st.markdown("#### Weaknesses")
                for c in sd["cons"]:
                    st.markdown(f"- {c}")
            else:
                st.markdown("#### Weaknesses")
                st.caption("None found on page.")

        # Quarterly financials
        if sd.get("quarterly_sales"):
            st.markdown("#### Quarterly Sales (₹ Cr)")
            periods = [p for p, _ in sd["quarterly_sales"]]
            values = [v.replace(",", "") for _, v in sd["quarterly_sales"]]
            try:
                fig_q = go.Figure(go.Bar(

```

```

        x=periods, y=[float(v) for v in values],
        marker_color="#00d4ff", text=values, textposition="outside"
    ))
    fig_q.update_layout(template="plotly_dark", height=300,
                        title="Quarterly Revenue", showlegend=False)
    st.plotly_chart(fig_q, use_container_width=True)
except Exception:
    st.table(sd["quarterly_sales"])

# Annual report PDF
if sd.get("annual_report_url"):
    st.markdown("#### Annual Report")
    st.markdown(f" [Download Annual Report PDF]({sd['annual_report_url']})")
    if st.session_state.pdf_summary:
        st.markdown("#### AI Summary of Annual Report")
        st.markdown(st.session_state.pdf_summary)
    else:
        if st.button(" Summarise Annual Report Now"):
            with st.spinner("Downloading & summarising..."):
                try:
                    summary = download_and_extract_pdf(sd["annual_report_url"], groq_key)
                    st.session_state.pdf_summary = summary
                    st.success("Done! Re-run analysis to include in RAG.")
                    st.markdown(summary)
                except Exception as e:
                    st.error(f"Failed: {e}")
else:
    st.info("No annual report PDF link found on Screener.in. You can upload one manually below.")
    uploaded = st.file_uploader("Upload Annual Report PDF (fallback)", type=["pdf"])
    if uploaded:
        with st.spinner("Extracting & summarising PDF..."):
            try:
                import pdfplumber, tempfile
                from groq import Groq
                tmp = tempfile.NamedTemporaryFile(suffix=".pdf", delete=False)
                tmp.write(uploaded.read()); tmp.flush()
                pages = []
                with pdfplumber.open(tmp.name) as pdf:
                    for i, page in enumerate(pdf.pages):
                        if i >= 20: break
                        txt = page.extract_text()
                        if txt: pages.append(txt)
                full_text = "\n\n".join(pages)
                client = Groq(api_key=groq_key)
                chunks_list = [full_text[i:i+15000] for i in range(0, min(len(full_text), 60000), 15000)]
                chunk_summaries = []
                for chunk in chunks_list:
                    resp = client.chat.completions.create(
                        model="llama-3.1-8b-instant",
                        messages=[{"role": "user", "content":
                                f"Summarise this annual report section in bullet points:\n\n{chunk}"
                                }],
                        temperature=0.2, max_tokens=512
                    )
                    chunk_summaries.append(resp.choices[0].message.content.strip())
                final_resp = client.chat.completions.create(
                    model="llama-3.1-8b-instant",
                    messages=[{"role": "user", "content":
                            "Combine into executive summary (Financial Performance, Risks, Outlook):\n\n" +
                            "\n---\n".join(chunk_summaries)
                            }],
                    temperature=0.2, max_tokens=800
                )
                st.session_state.pdf_summary = final_resp.choices[0].message.content.strip()
                st.success("PDF summarised! Re-run analysis to include in RAG.")
                st.markdown(st.session_state.pdf_summary)
            except Exception as e:
                st.error(f"PDF processing failed: {e}")

# RAG CHATBOT
with tab6:
    st.subheader(" RAG Chatbot")
    cn = st.session_state.company_name or "the company"

    if not st.session_state.models_trained:
        st.info("Run analysis first to initialise the knowledge base.")
    else:
        doc_count = len(st.session_state.documents)

```

```

has_screener = bool(st.session_state.get("screener_data", {}).get("raw_text"))
has_pdf      = bool(st.session_state.pdf_summary)

st.caption(
    f"Knowledge base: **{doc_count} chunks** - "
    f"{'Screener data' if has_screener else 'No Screener data'} . "
    f"{' Annual report' if has_pdf else 'No annual report'} . "
    f"{' News headlines' if st.session_state.headlines else 'No news'}"
)

for msg in st.session_state.chat_history:
    if msg["role"] == "user":
        st.markdown(f'<div class="chat-user"> 🗨️ {msg["content"]}</div>',
                    unsafe_allow_html=True)
    else:
        st.markdown(f'<div class="chat-bot"> 🤖 {msg["content"]}</div>',
                    unsafe_allow_html=True)

with st.form("chat_form", clear_on_submit=True):
    user_q = st.text_input("Your question", placeholder="What are the key risks for this company?")
    submitted = st.form_submit_button("Send ➡️")

if submitted and user_q.strip():
    st.session_state.chat_history.append({"role": "user", "content": user_q})
    with st.spinner("Thinking..."):
        try:
            ans = ask_rag(user_q, st.session_state.embedder, st.session_state.faiss_index,
                          st.session_state.documents, st.session_state.groq_client, cn)
        except Exception as e:
            ans = f"Error: {e}"
    st.session_state.chat_history.append({"role": "assistant", "content": ans})
    st.rerun()

if st.session_state.chat_history:
    if st.button("🗑️ Clear chat"):
        st.session_state.chat_history = []
        st.rerun()

st.markdown("#### Quick Questions")
qqs = [
    "What are the key risks?",
    "What are the growth drivers?",
    "What do the key ratios say about valuation?",
    "Summarise the latest news sentiment.",
    "Is the company financially healthy?",
    "What is the revenue trend?",
]

cols = st.columns(2)
for i, qq in enumerate(qqs):
    if cols[i % 2].button(qq, key=f"qq_{i}"):
        st.session_state.chat_history.append({"role": "user", "content": qq})
        with st.spinner("Thinking..."):

```

Overwriting app.py

```

!pip install pyngrok -q
!ngrok authtoken 3AP0v0sN8iv5eXEm7ApEivYe8zF 7RixYsJBRqV8JaZdgtuXU

```